SLIIT UNI

THE KNOWLEDGE UNIVERSITY

07 – Transport Layer

IE2004 Computer Networks

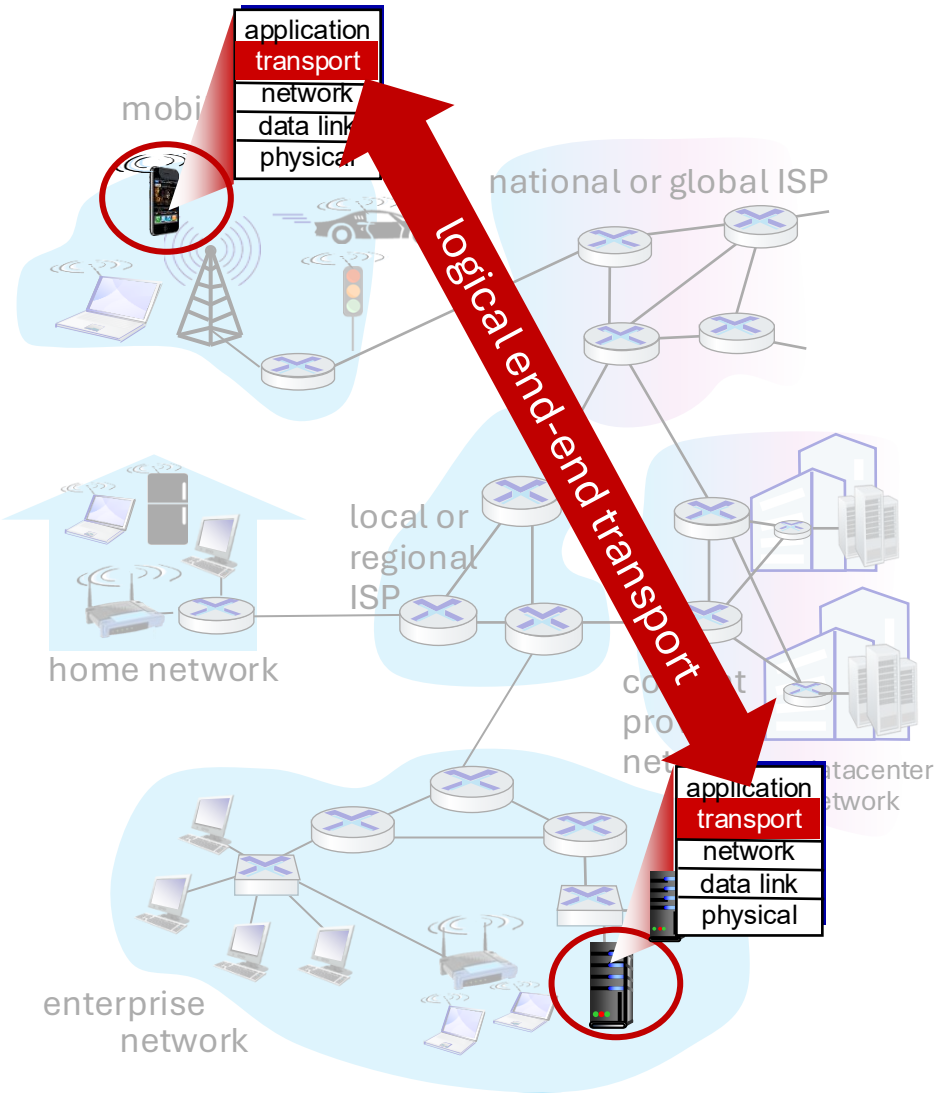
2026 July

Lesson outline

- Transport-layer services
- Multiplexing and demultiplexing
- Connectionless transport: UDP
- Connection-oriented transport: TCP
- TCP connection management
- TCP Sequence numbers and acknowledgements
- TCP Retransmission
- TCP Flow control
- TCP Congestion control

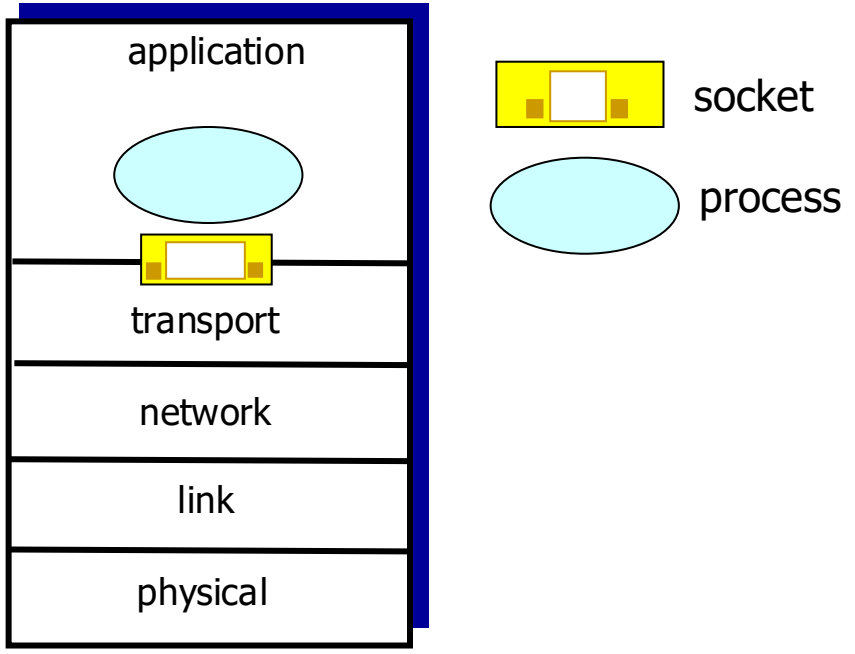
Transport services and protocols

- provide logical communication between application processes running on different hosts
- transport protocols actions in end systems:
  - sender: breaks application messages into segments, passes to network layer
  - receiver: reassembles segments into messages, passes to application layer
- two transport protocols available to Internet applications
  - TCP, UDP



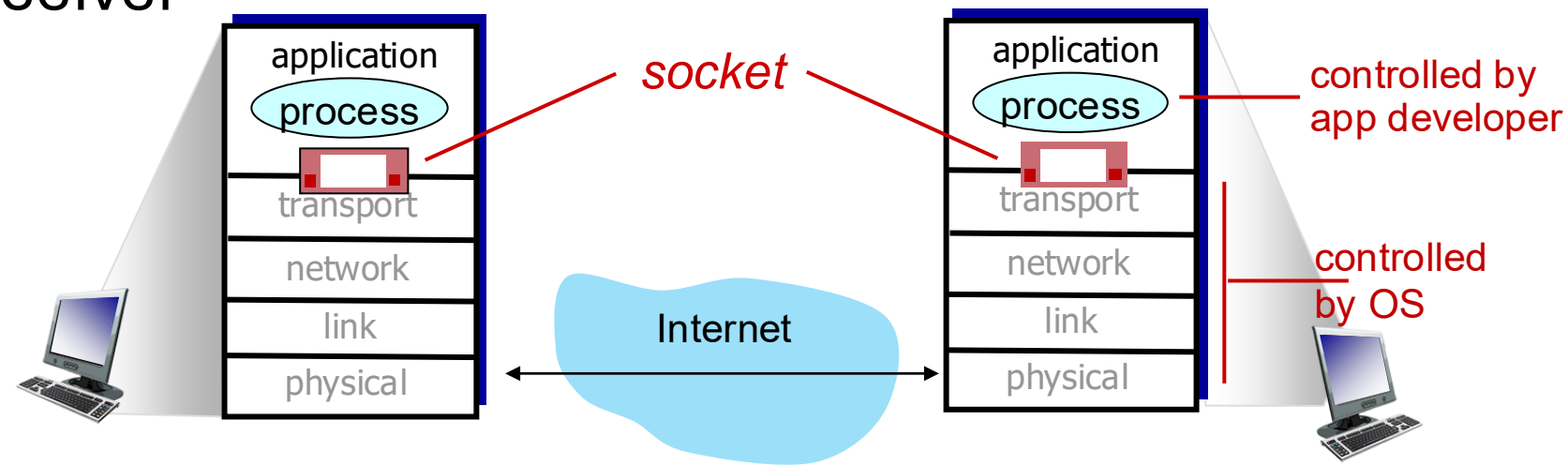
Sockets

- A process in the Application Layer connects to the Transport Layer via a socket
  - Process = running app / part of a running app
- A socket is an endpoint for sending or receiving data across a network
- Socket created by application layer process
  - socket is connected to that process
- A socket is identified by the protocol, IP address and port number
  - Plus remote IP and port number in TCP
  - More about this later in the lesson



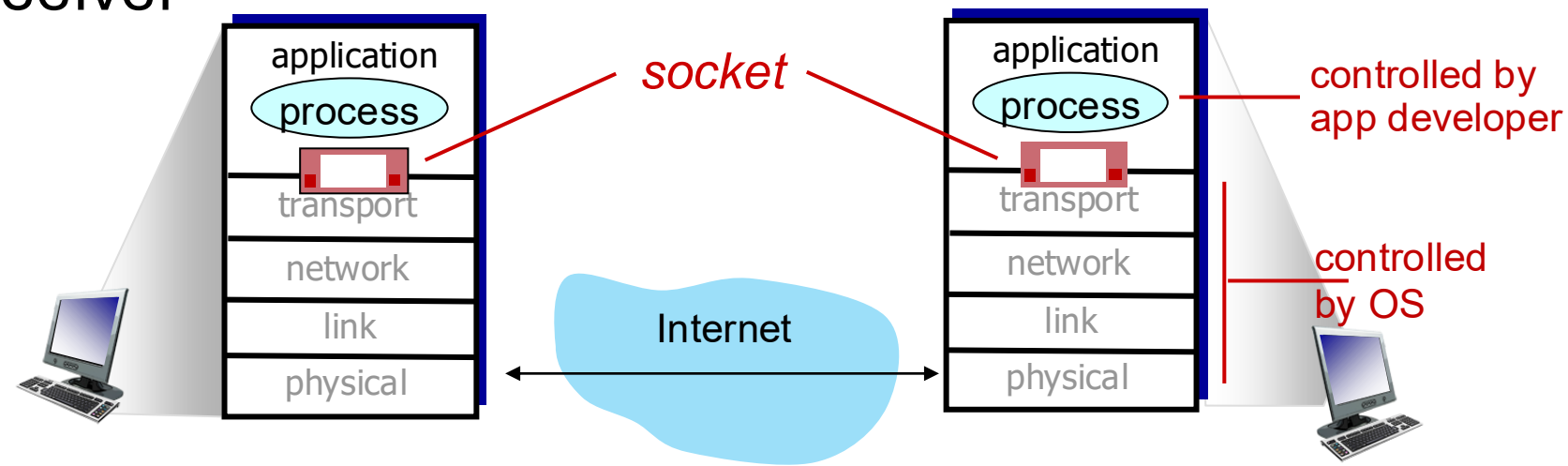
Sockets (ctd)

- A socket is like a door
  - process sends/receives messages to/from its socket
  - sending process pushes message out through the door
  - sending process relies on transport layer on other side of door to deliver message to socket at receiving process
  - communication involves two sockets: one at sender and one at receiver

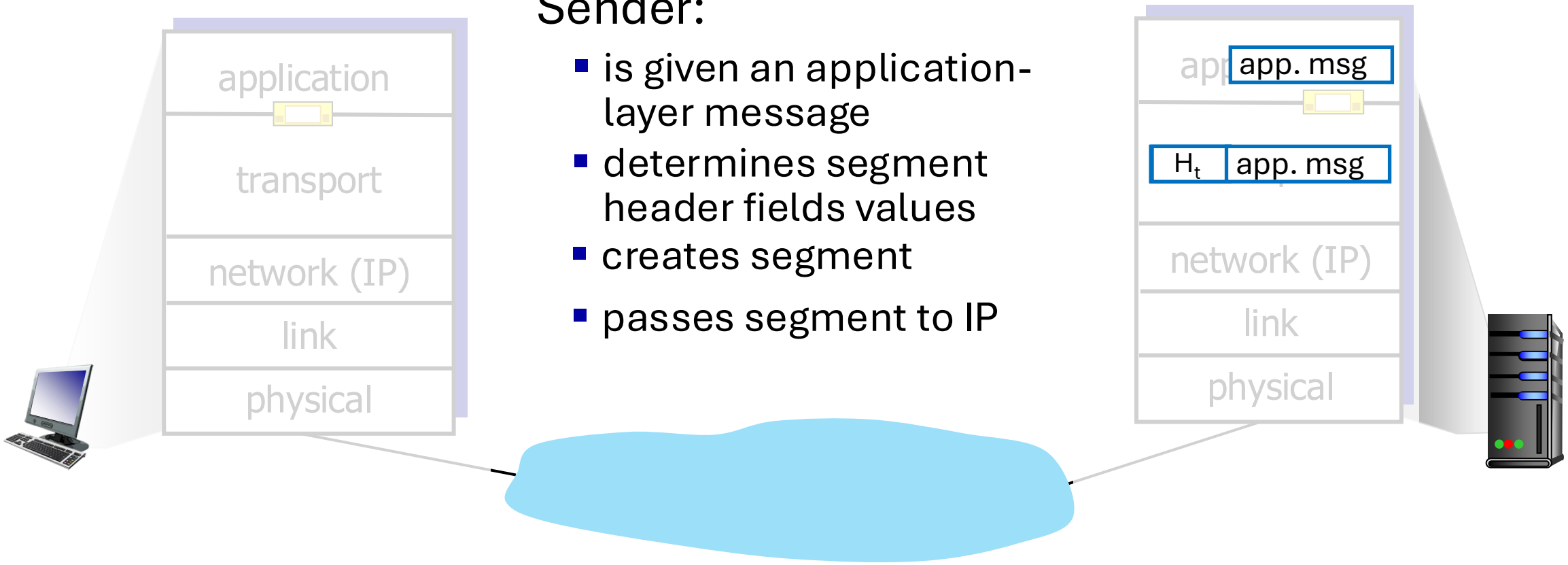


Sockets (ctd)

- A socket is like a door
  - process sends/receives messages to/from its socket
  - sending process pushes message out through the door
  - sending process relies on transport layer on other side of door to deliver message to socket at receiving process
  - communication involves two sockets: one at sender and one at receiver



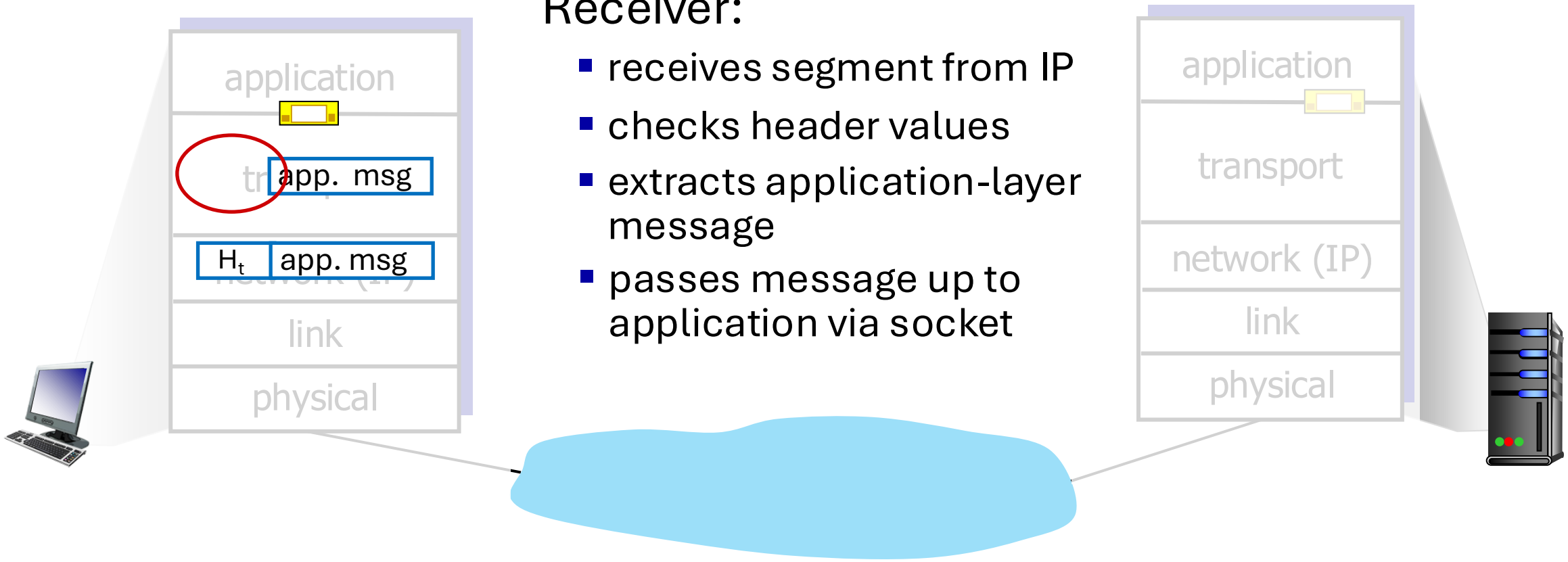
Transport Layer Actions



Sender:

- is given an application-layer message
- determines segment header fields values
- creates segment
- passes segment to IP

Transport Layer Actions



Receiver:

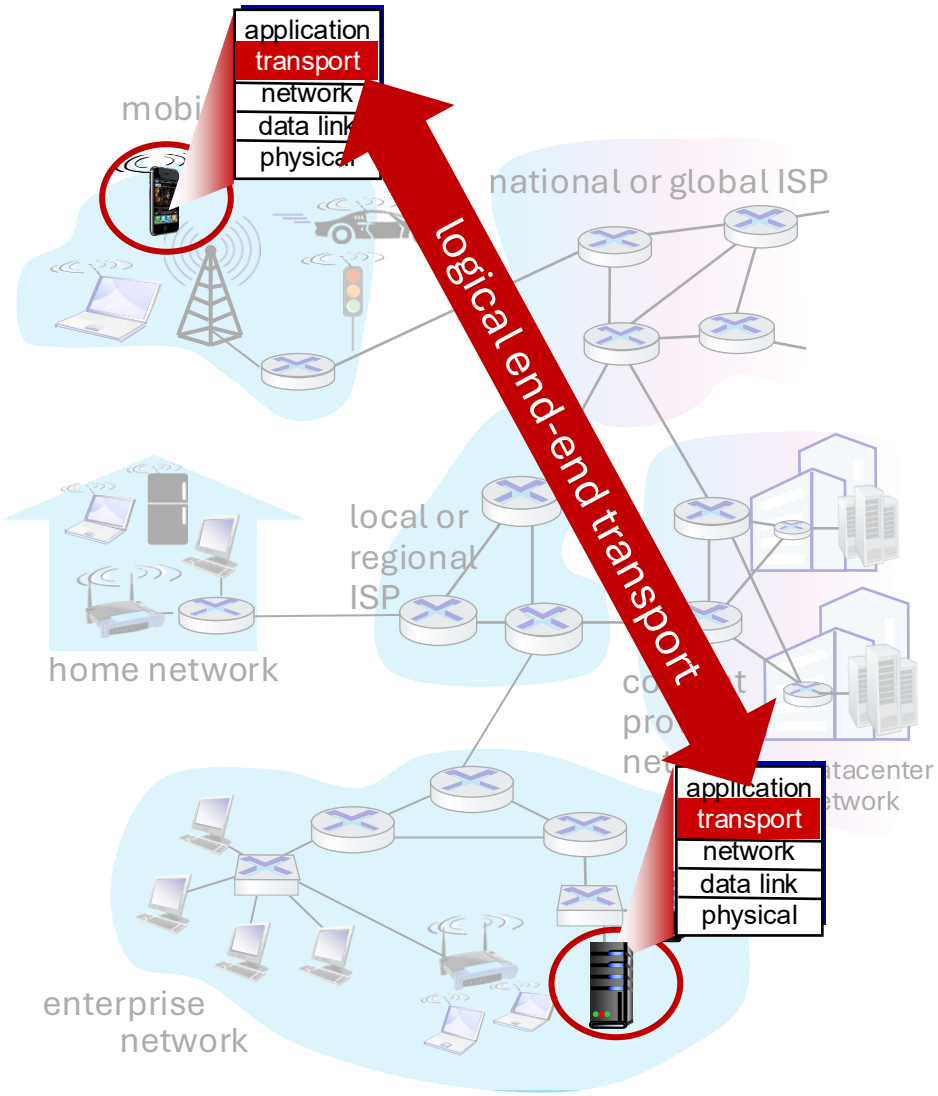
- receives segment from IP
- checks header values
- extracts application-layer message
- passes message up to application via socket

Transport layer / TCP and UDP

- Datagram delivery system in the Network Layer (L3 or IP layer) is
  - Unreliable: Delivery is not guaranteed
  - Best effort: Some datagrams may be dropped due to congestion etc
  - Connectionless: Each datagram is treated independently from others
- What if applications needs reliability?
- Transport layer has two protocols
  - TCP: Connection oriented, Reliable (does extra work to provide reliability)
  - UDP: Connectionless, Same unreliability as IP (doesn't do any extra work)
- As there may be multiple applications running on a single host, we need a way to identify each application / reach them over Internet
  - Just the IP address is not sufficient
  - Transport layer identifies each app using a Port number (16-bit number)

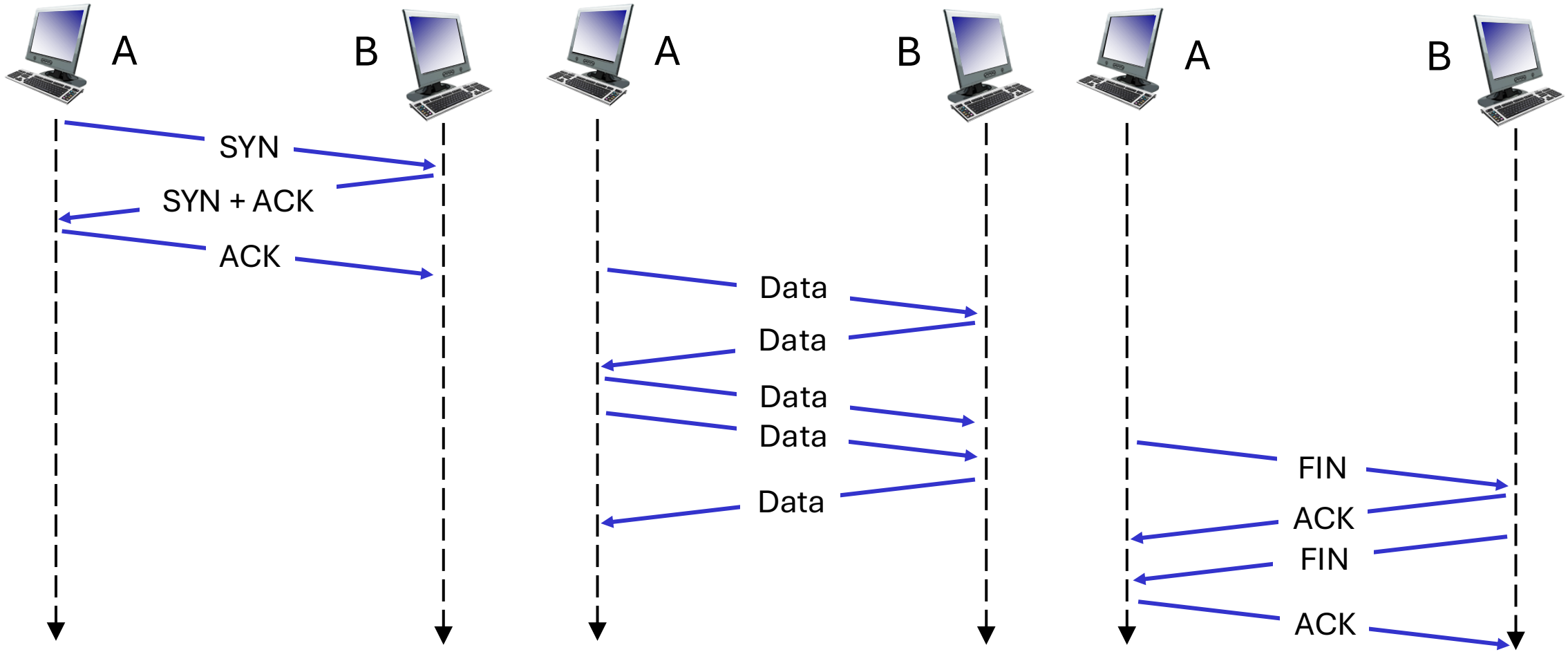
Two principal Internet transport protocols

- TCP: Transmission Control Protocol
  - connection setup, release
  - reliable, in-order delivery
  - congestion control
  - flow control
- UDP: User Datagram Protocol
  - connectionless
  - unreliable, unordered delivery
  - no-frills extension of “best-effort” IP
- services not available:
  - delay guarantees
  - bandwidth guarantees



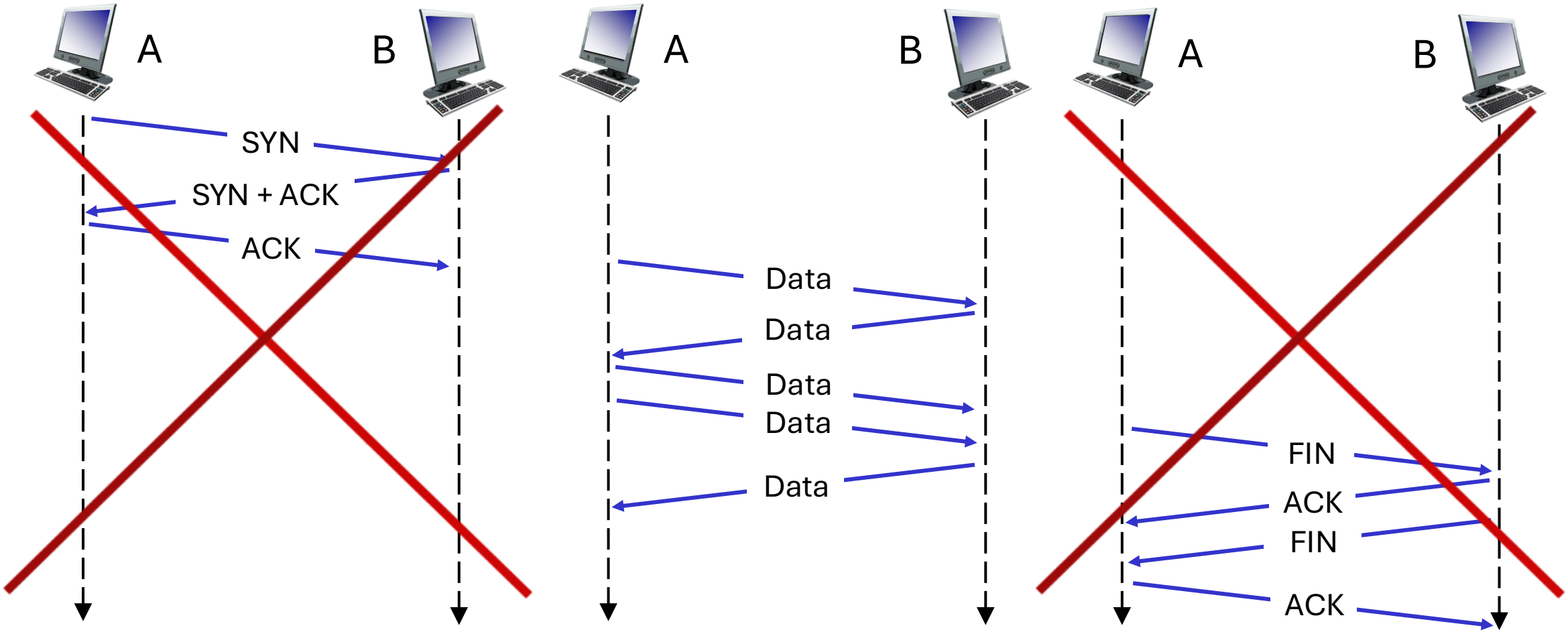
TCP: Connection-oriented, Reliable

- Connection establishment, Data transfer, Connection closure



UDP: Connectionless, Same unreliability as IP

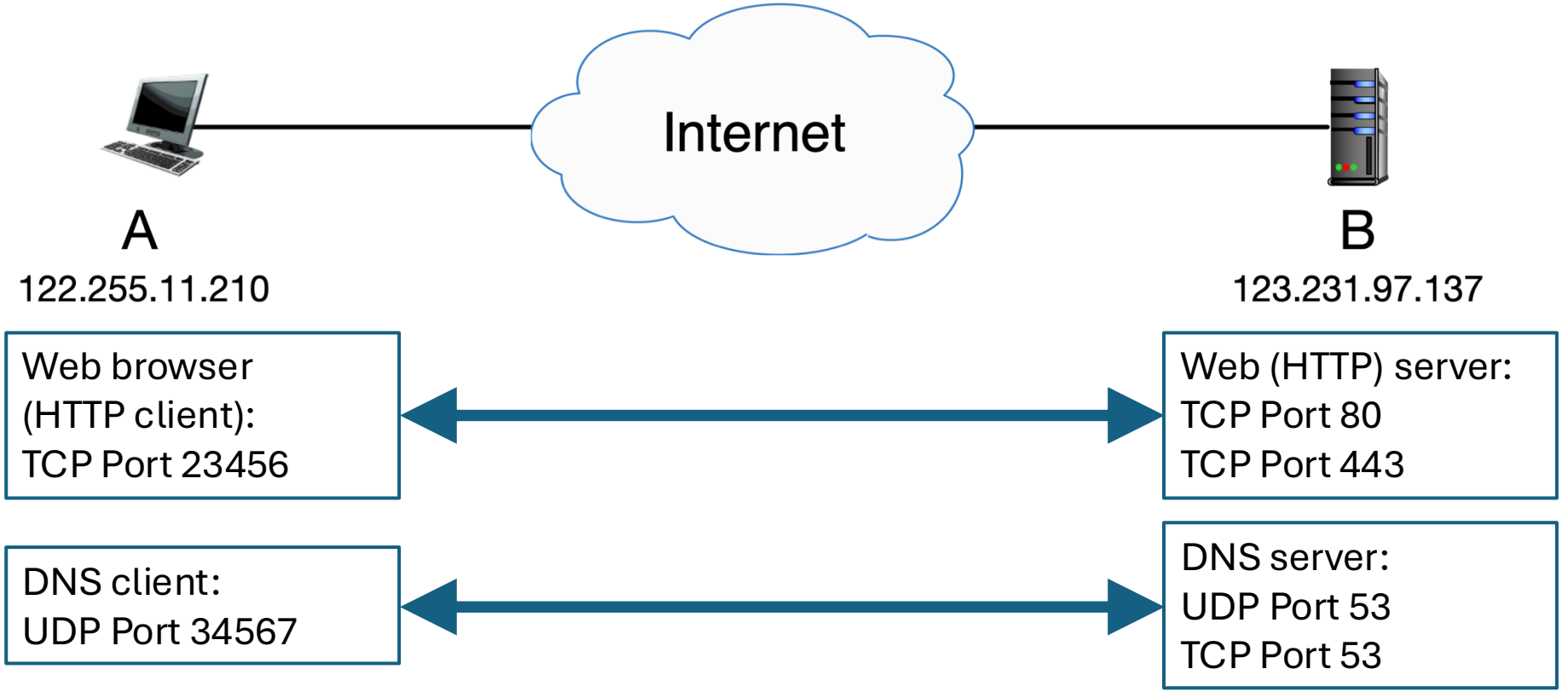
- No connection establishment or Connection closure



Lesson outline

- Transport-layer services
- Multiplexing and demultiplexing
- Connectionless transport: UDP
- Connection-oriented transport: TCP
- TCP connection management
- TCP Sequence numbers and acknowledgements
- TCP Retransmission
- TCP Flow control
- TCP Congestion control

Applications and Port numbers

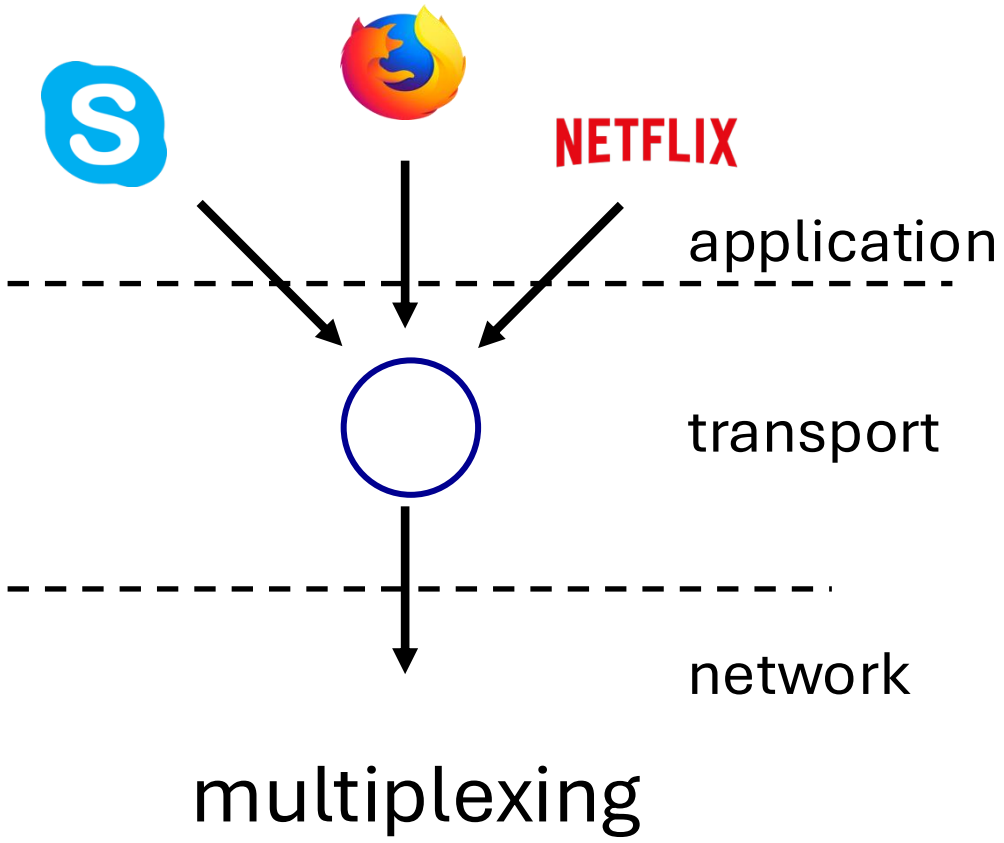


- Each application can be uniquely identified & reached over network
  - Web browser is 122.255.11.210:23456 (TCP)
  - DNS client is 122.255.11.210:34567 (UDP)
  - Web server is 123.231.97.137:80 (TCP) and 123.231.97.137:443 (TCP)
  - DNS server is 123.231.97.137:53 (TCP) and 123.231.97.137:53 (UDP)

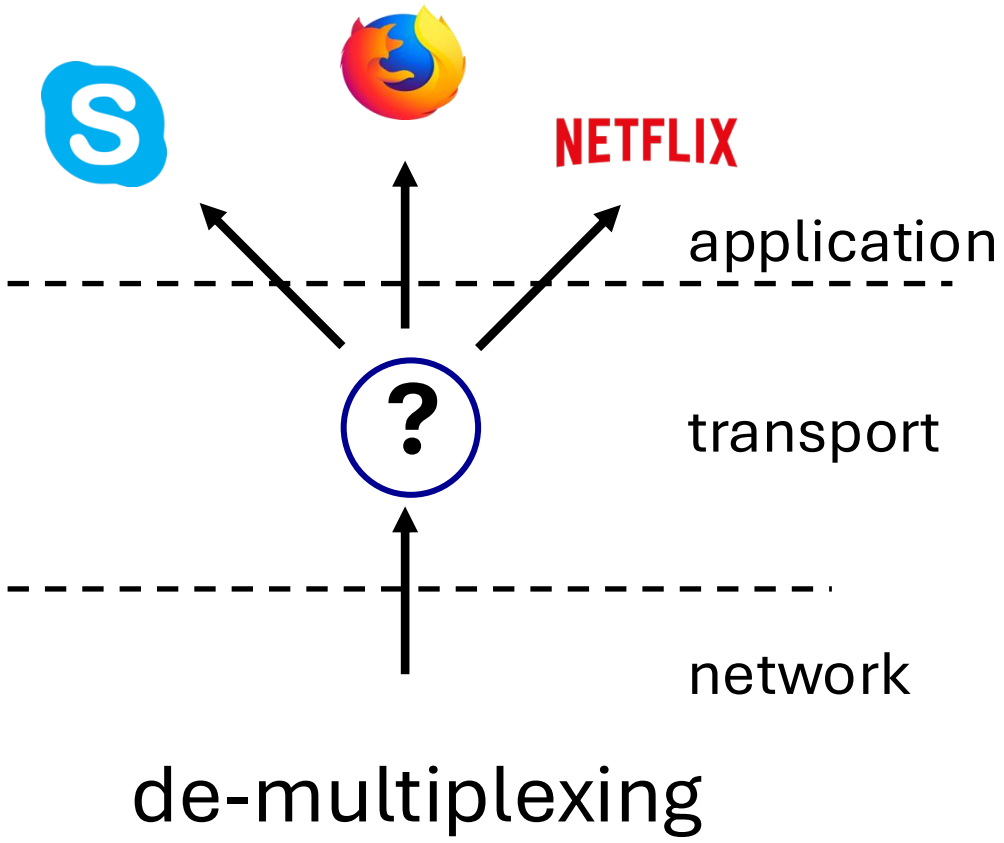
Multiplexing / Demultiplexing

- How to identify different applications running on the same host?
  - Network layer (L3) address of the host is common to all apps
- Transport layer provides a port number, which can be used to identify each application (each app. uses a different port number)
- When sending, segments related to different applications using different ports are combined (multiplexed) into a single data stream and passed to the network layer
- When receiving, received segments are distributed (demultiplexed) to the correct application via the correct socket
  - using destination port number in the segment to identify correct socket

Multiplexing



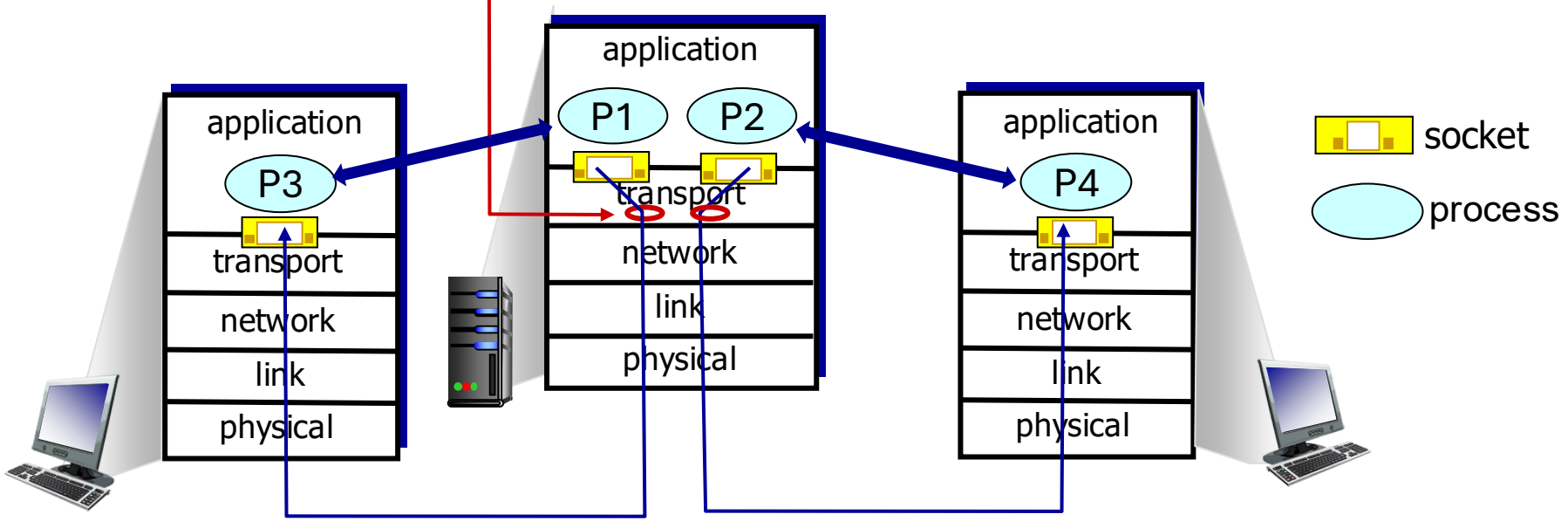
De-multiplexing



Multiplexing as sender

multiplexing as sender:

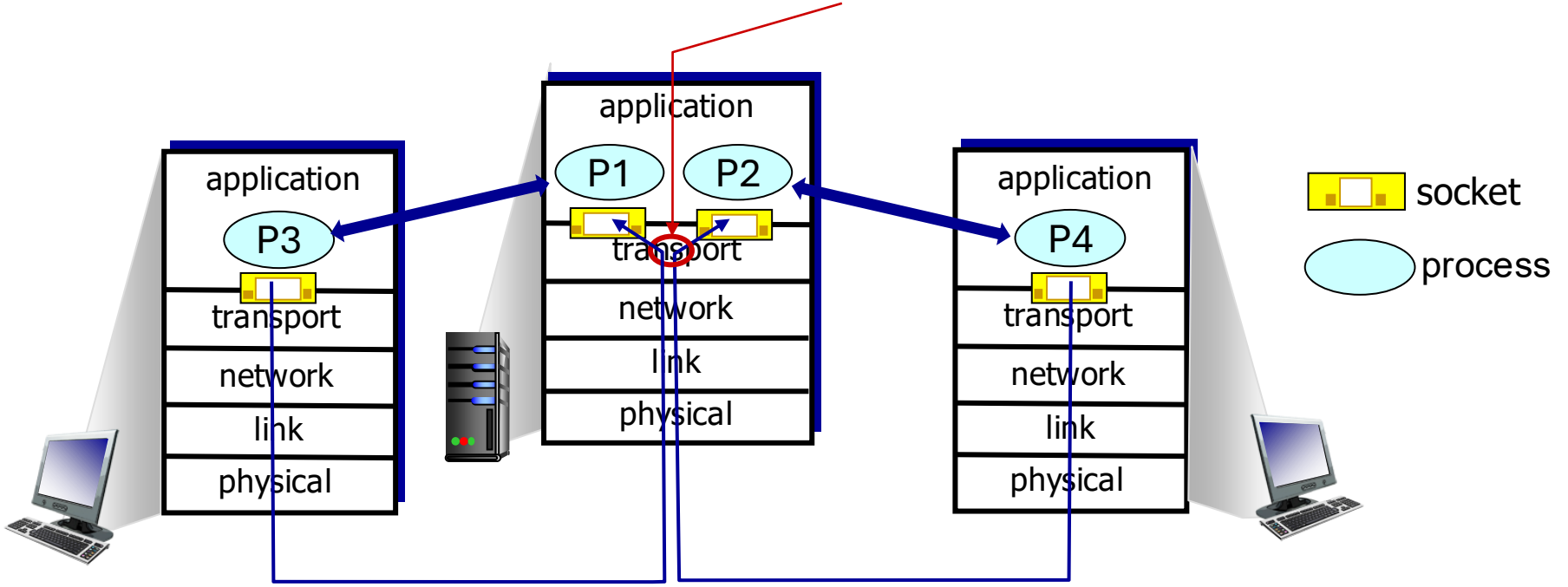
handle data from multiple sockets, add transport layer header, and pass to layer 3

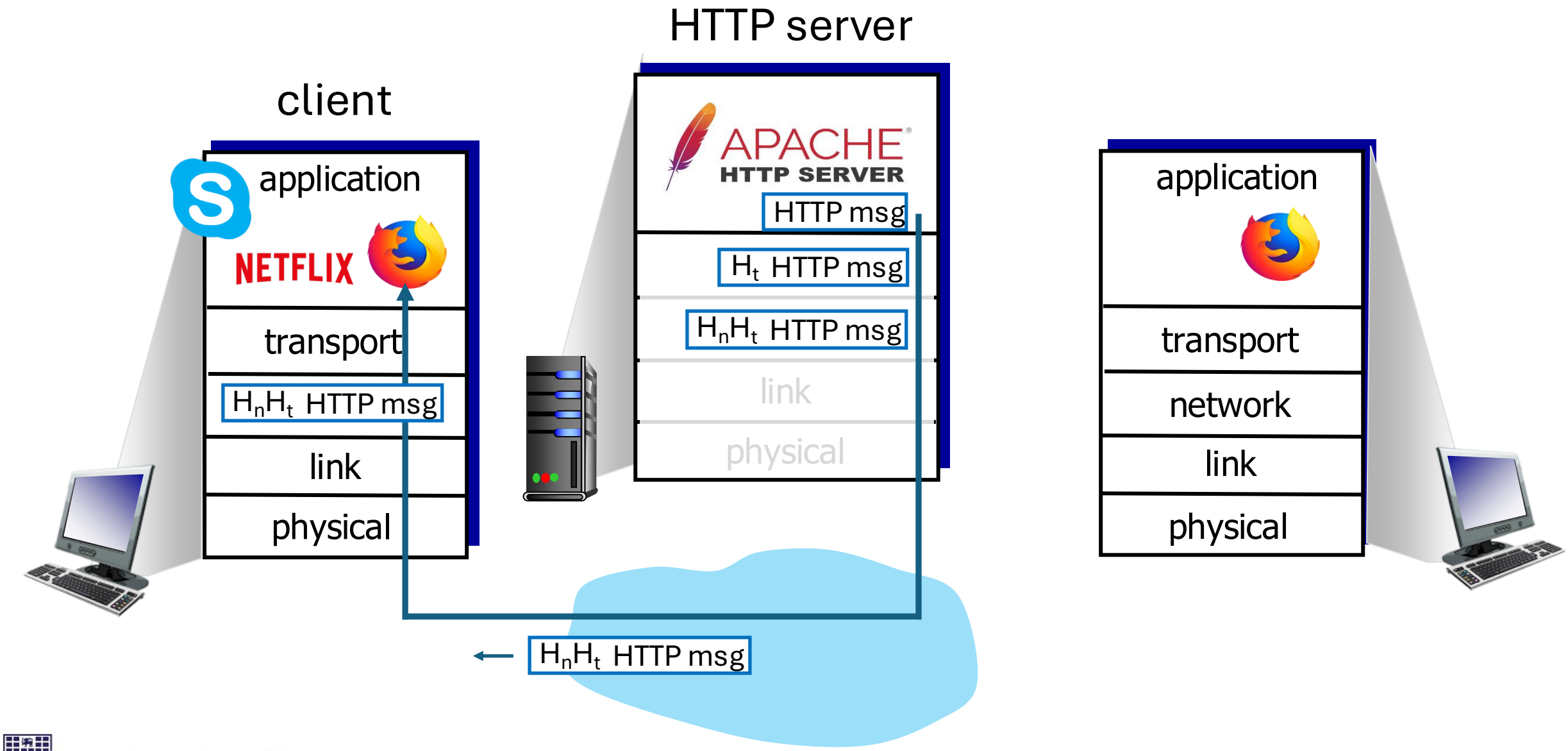


Demultiplexing as receiver

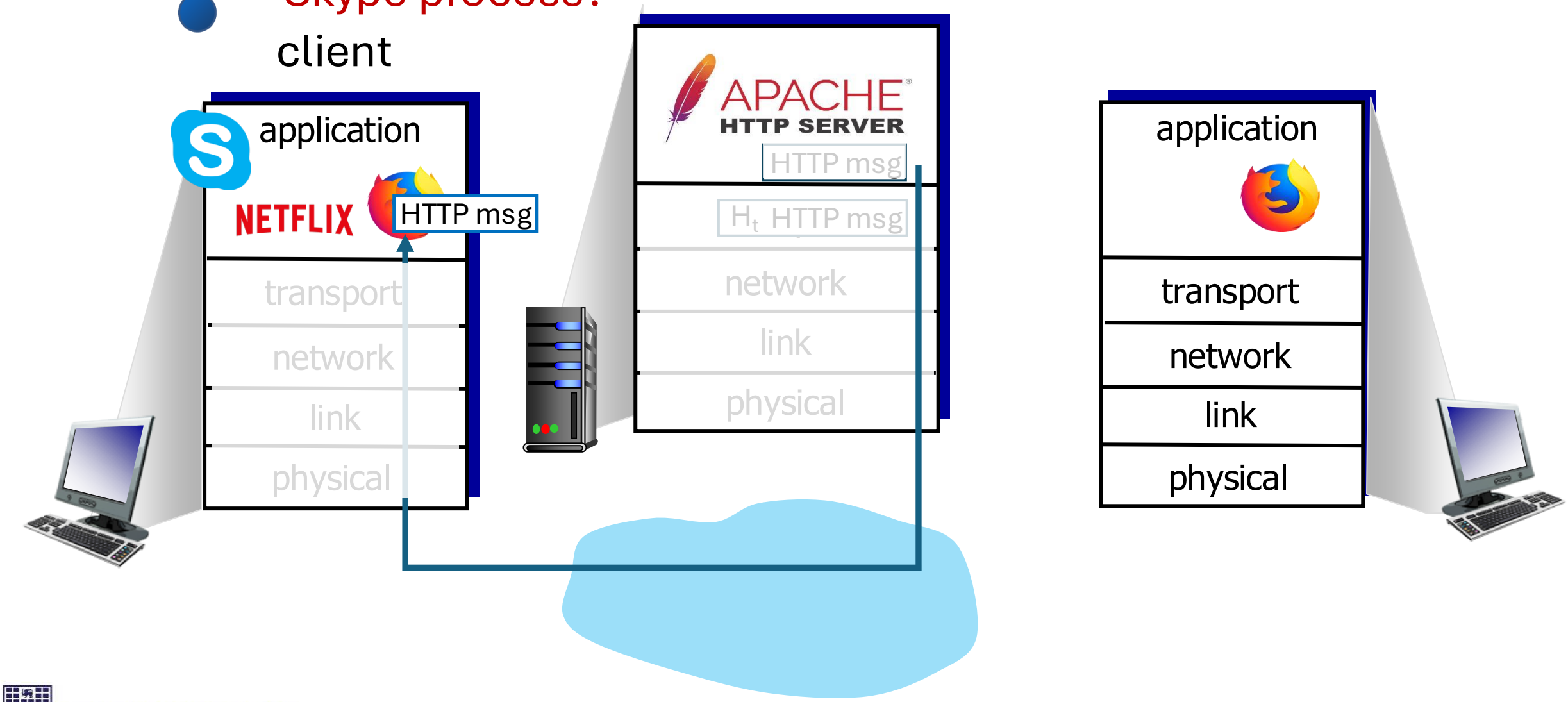
demultiplexing as receiver:

use destination port number in header to deliver received segments to correct socket



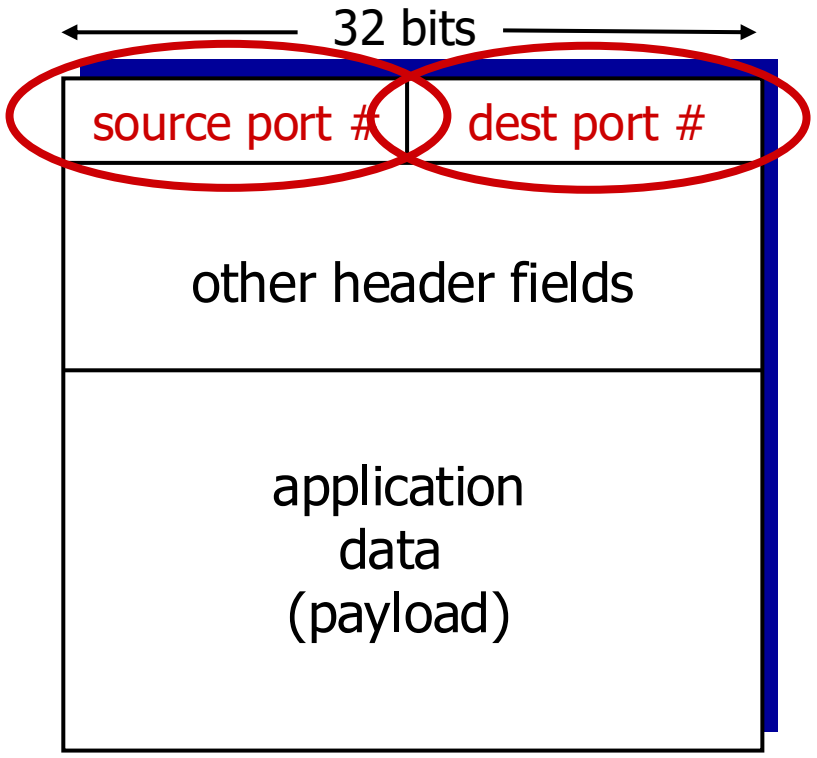

HTTP server
client
application
NETFLIX
transport
H_nH_t HTTP msg
link
physical
APACHE
HTTP SERVER
HTTP msg
H_t HTTP msg
H_nH_t HTTP msg
link
physical
H_nH_t HTTP msg
application
transport
network
link
physical

Q: how did transport layer at client know to deliver message to Firefox browser process rather than Netflix process or Skype process?



How demultiplexing works

- host receives IP datagrams
  - each datagram has source IP address, destination IP address
  - each datagram carries one transport-layer segment
  - each segment has source and destination port numbers
- host uses IP addresses & port numbers to direct segment to appropriate socket

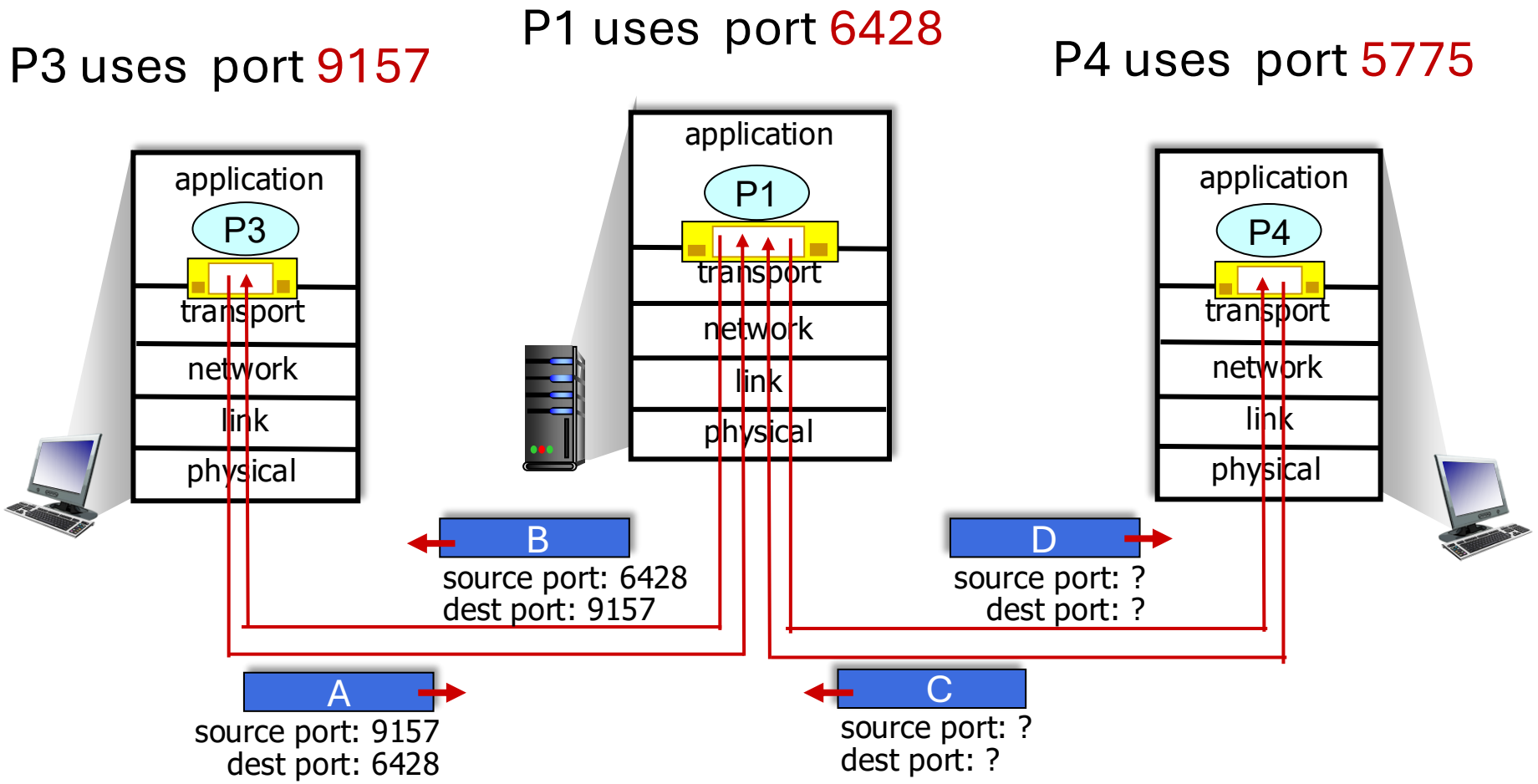


TCP/UDP segment format

Connectionless demultiplexing

- when receiving host receives UDP segment:
  - checks destination port number in segment
  - directs UDP segment to socket with that port number
- UDP datagrams will be delivered into socket with the matching destination port number
  - irrespective of the source IP address or source port number
  - there will be only one socket using a particular port number

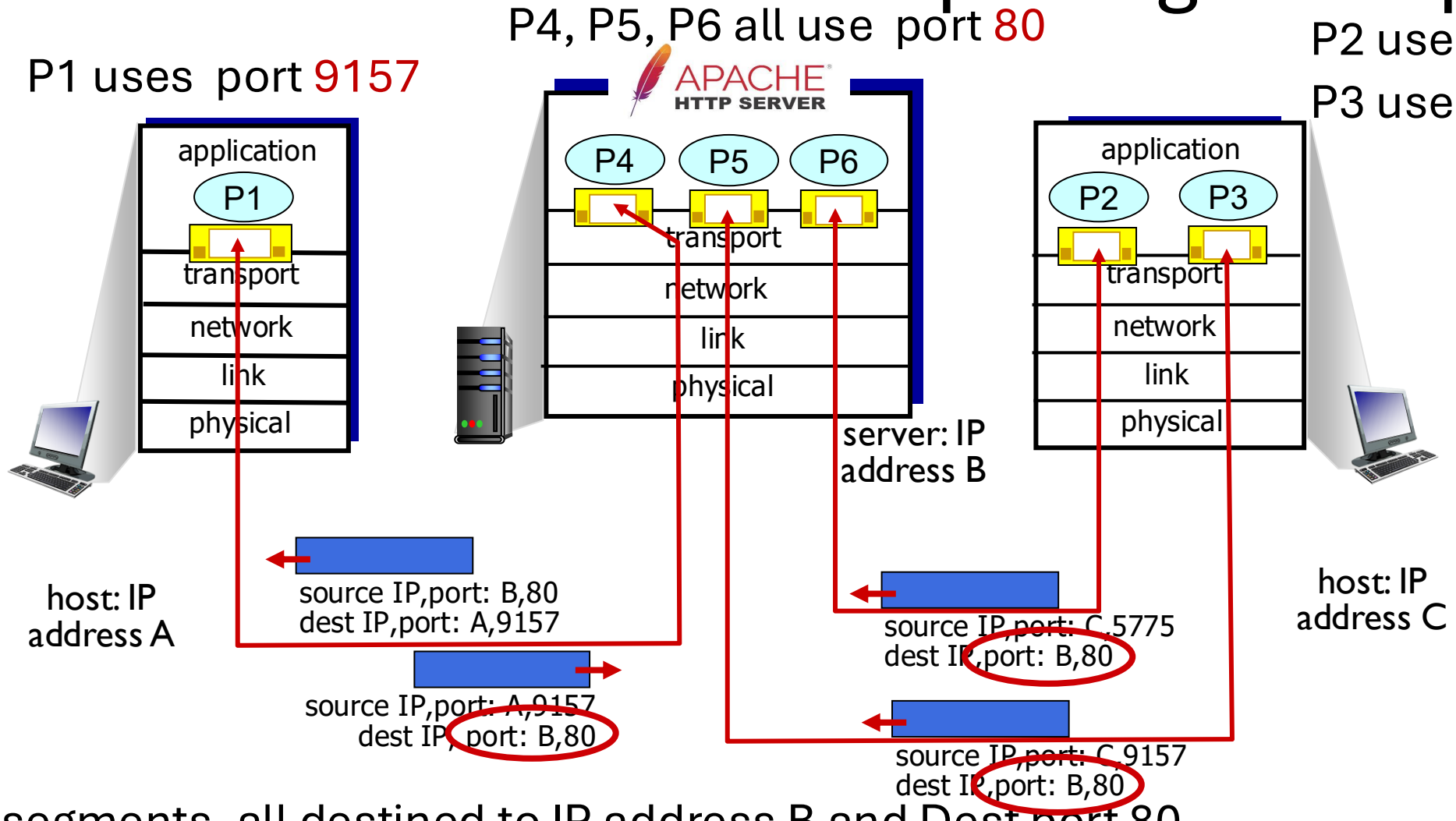
Connectionless demultiplexing: an example



Connection-oriented demultiplexing

- during connection setup, a new socket is created by server for every new client
- server may have many simultaneous TCP sockets
- server side IP and port number same for all such sockets
- each socket associated with a different connecting client
- each socket identified by its own 4-tuple
- TCP socket identified by 4-tuple:
  - source IP address
  - source port number
  - destination IP address
  - destination port number
- demultiplexing: receiver can't identify the correct socket using the destination port number only
- demultiplexing: receiver uses all four values in the 4-tuple to identify the correct socket to deliver the segment

Connection-oriented demultiplexing: example



Three segments, all destined to IP address B and Dest port 80 are demultiplexed to different sockets, based on source IP and Port number

Summary

- Multiplexing, demultiplexing: based on segment, datagram header field values
- UDP: demultiplexing using destination port number (only)
- TCP: demultiplexing using 4-tuple: source and destination IP addresses, and port numbers

Lesson outline

- Transport-layer services
- Multiplexing and demultiplexing
- Connectionless transport: UDP
- Connection-oriented transport: TCP
- TCP connection management
- TCP Sequence numbers and acknowledgements
- TCP Retransmission
- TCP Flow control
- TCP Congestion control

UDP: User Datagram Protocol

- "no frills," "bare bones" Internet transport protocol
- "best effort" service, UDP segments may be:
  - lost
  - delivered out-of-order to app
- connectionless:
  - no handshaking between UDP sender, receiver
  - each UDP segment handled independently of others

Why is there a UDP?

- no connection establishment (which can add RTT delay)
- simple: no connection state at sender, receiver
- small header size
- no congestion control
  - UDP can blast away as fast as desired!
  - can function in the face of congestion

UDP: User Datagram Protocol

- UDP use:
  - streaming multimedia apps (loss tolerant, rate sensitive)
  - DNS
  - SNMP
  - HTTP/3
- if reliable transfer needed over UDP (e.g., HTTP/3):
  - add needed reliability at application layer
  - add congestion control at application layer

UDP: User Datagram Protocol [RFC 768]

INTERNET STANDARD

RFC 768

J. Postel
ISI
28 August 1980

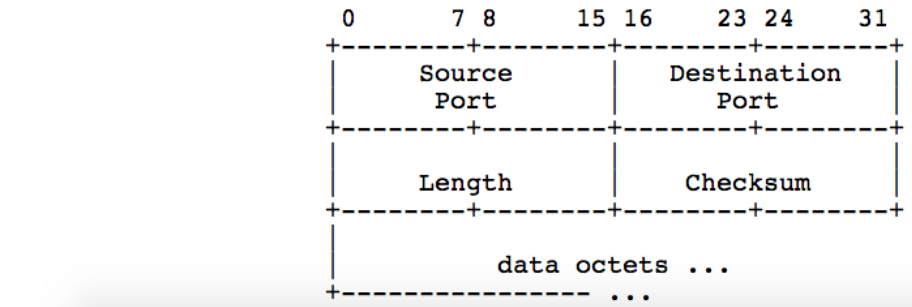
User Datagram Protocol

Introduction

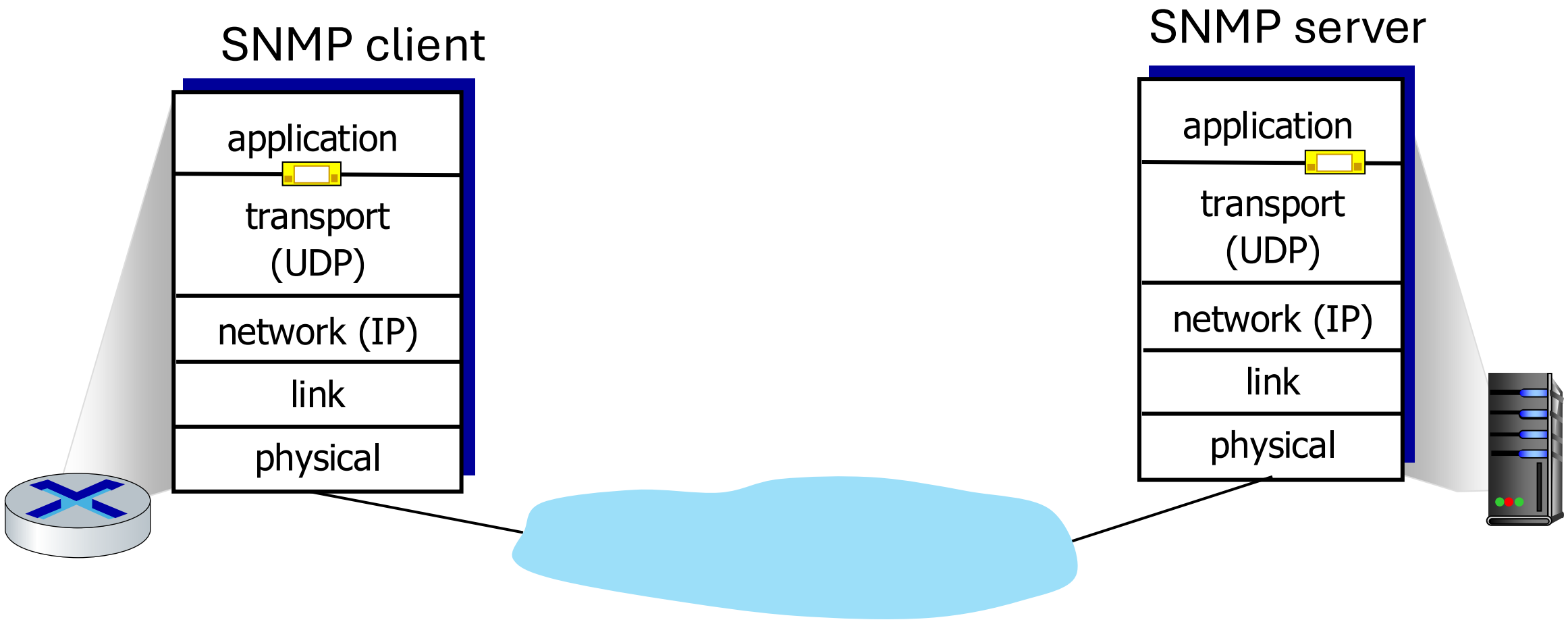
This User Datagram Protocol (UDP) is defined to make available a datagram mode of packet-switched computer communication in the environment of an interconnected set of computer networks. This protocol assumes that the Internet Protocol (IP) [1] is used as the underlying protocol.

This protocol provides a procedure for application programs to send messages to other programs with a minimum of protocol mechanism. The protocol is transaction oriented, and delivery and duplicate protection are not guaranteed. Applications requiring ordered reliable delivery of streams of data should use the Transmission Control Protocol (TCP) [2].

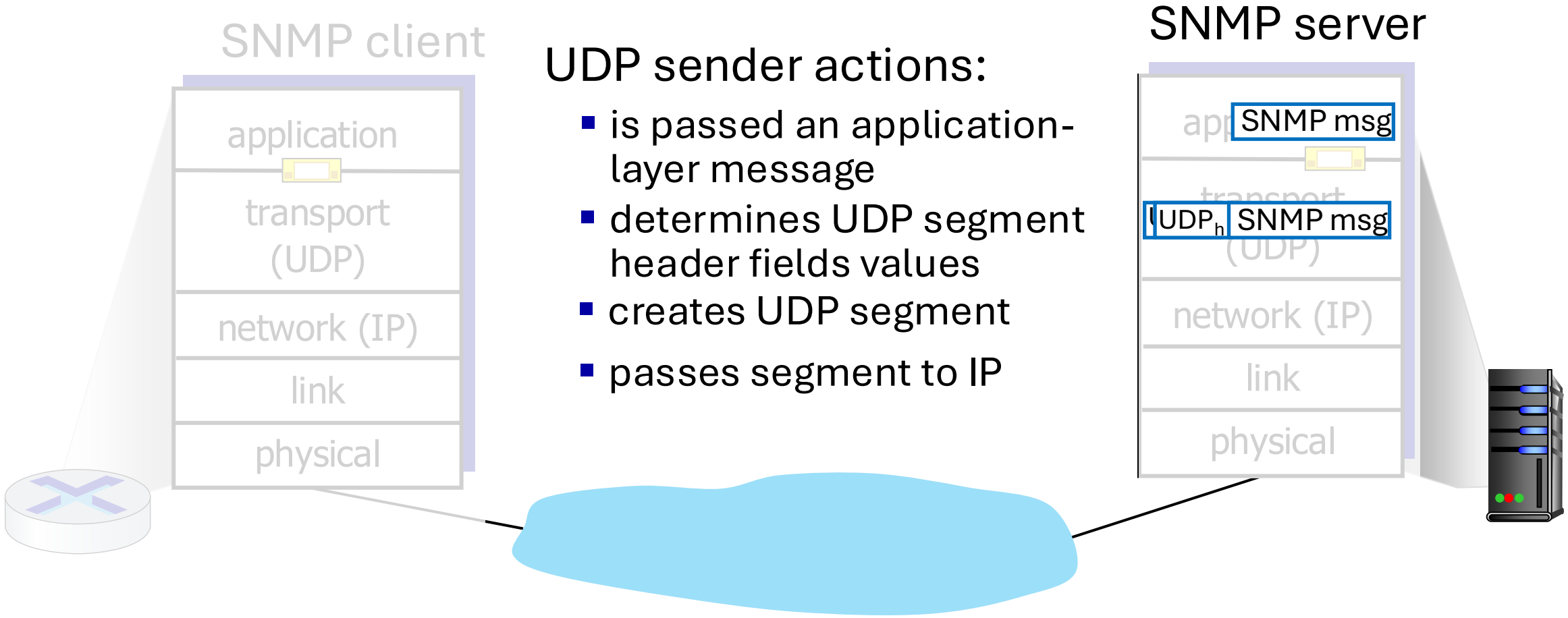
Format



UDP: Transport Layer Actions



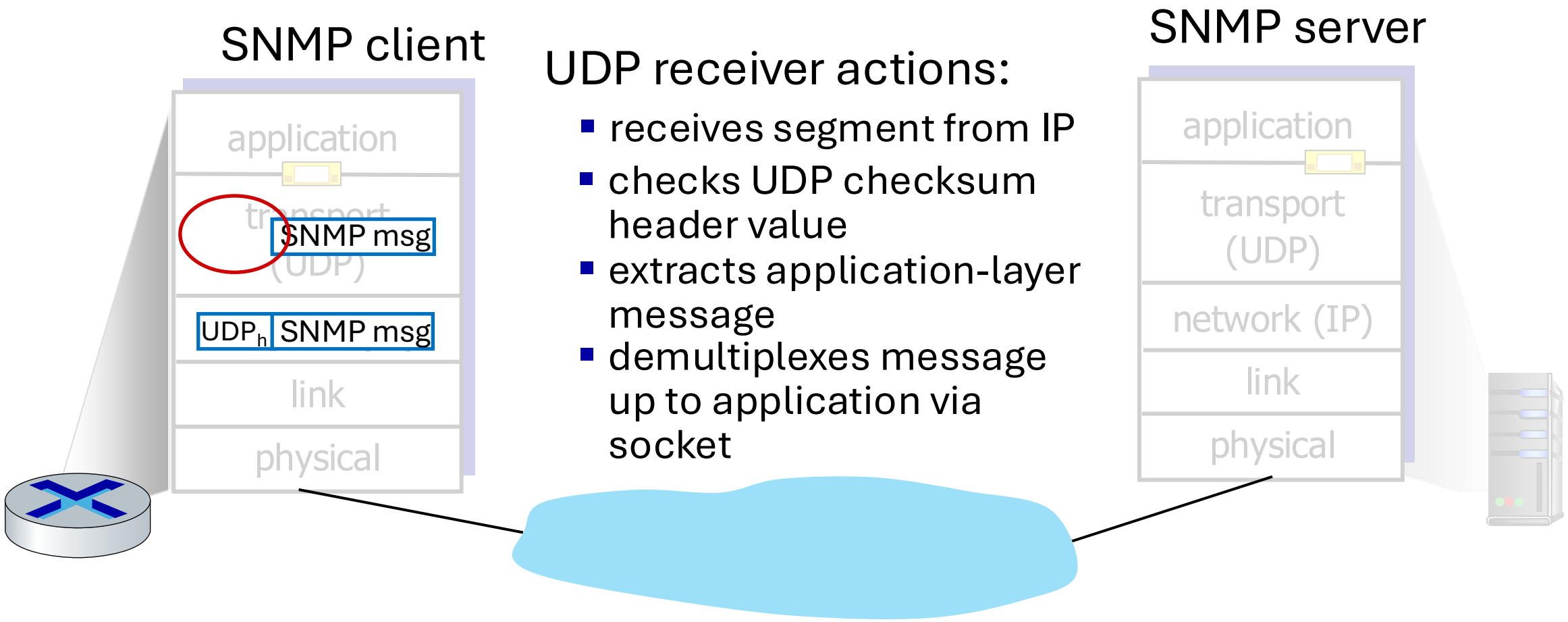
UDP: Transport Layer Actions



UDP sender actions:

- is passed an application-layer message
- determines UDP segment header fields values
- creates UDP segment
- passes segment to IP

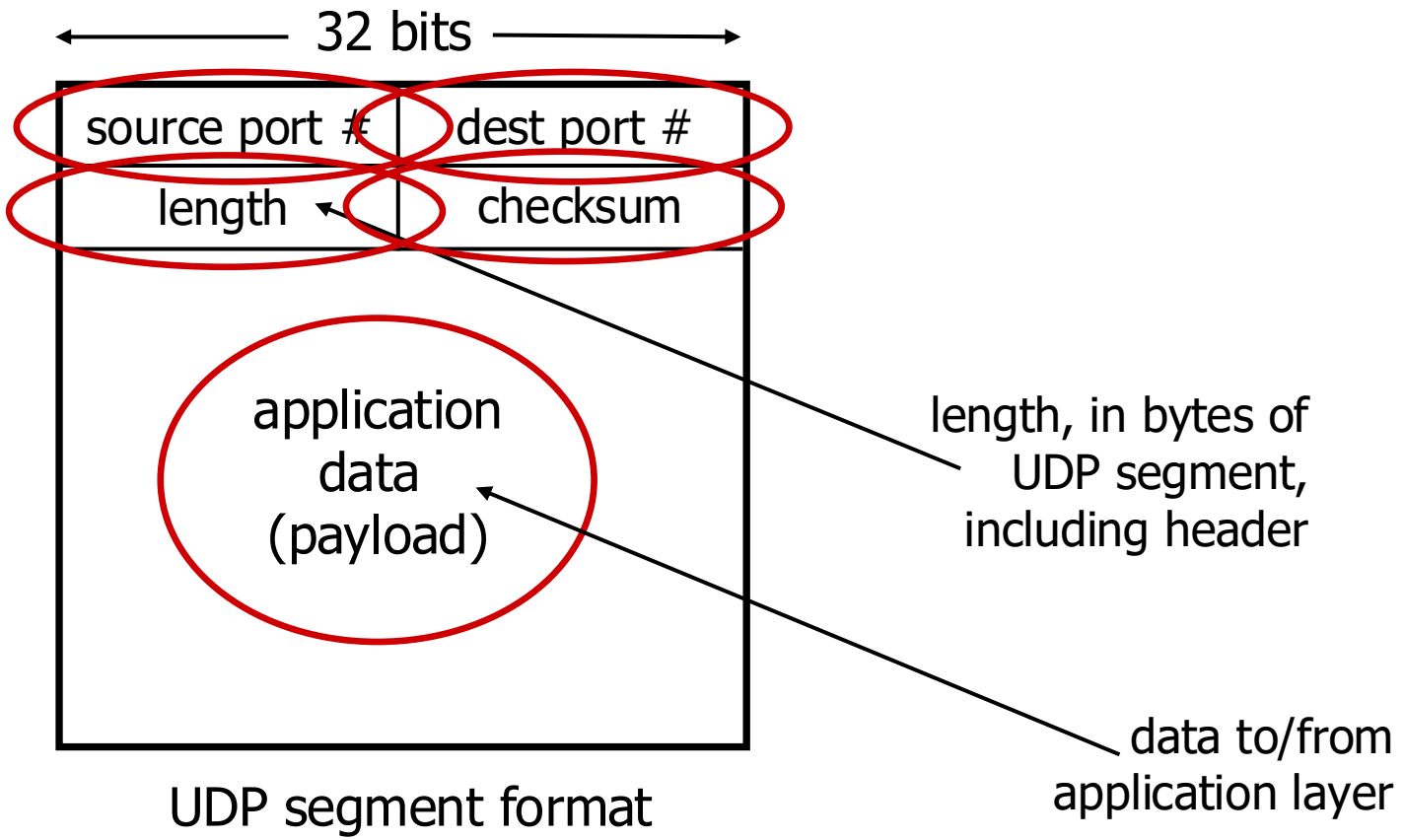
UDP: Transport Layer Actions



UDP receiver actions:

- receives segment from IP
- checks UDP checksum header value
- extracts application-layer message
- demultiplexes message up to application via socket

UDP segment header



UDP checksum (Internet checksum)

Goal: detect errors (i.e., flipped bits) in transmitted segment

sender:

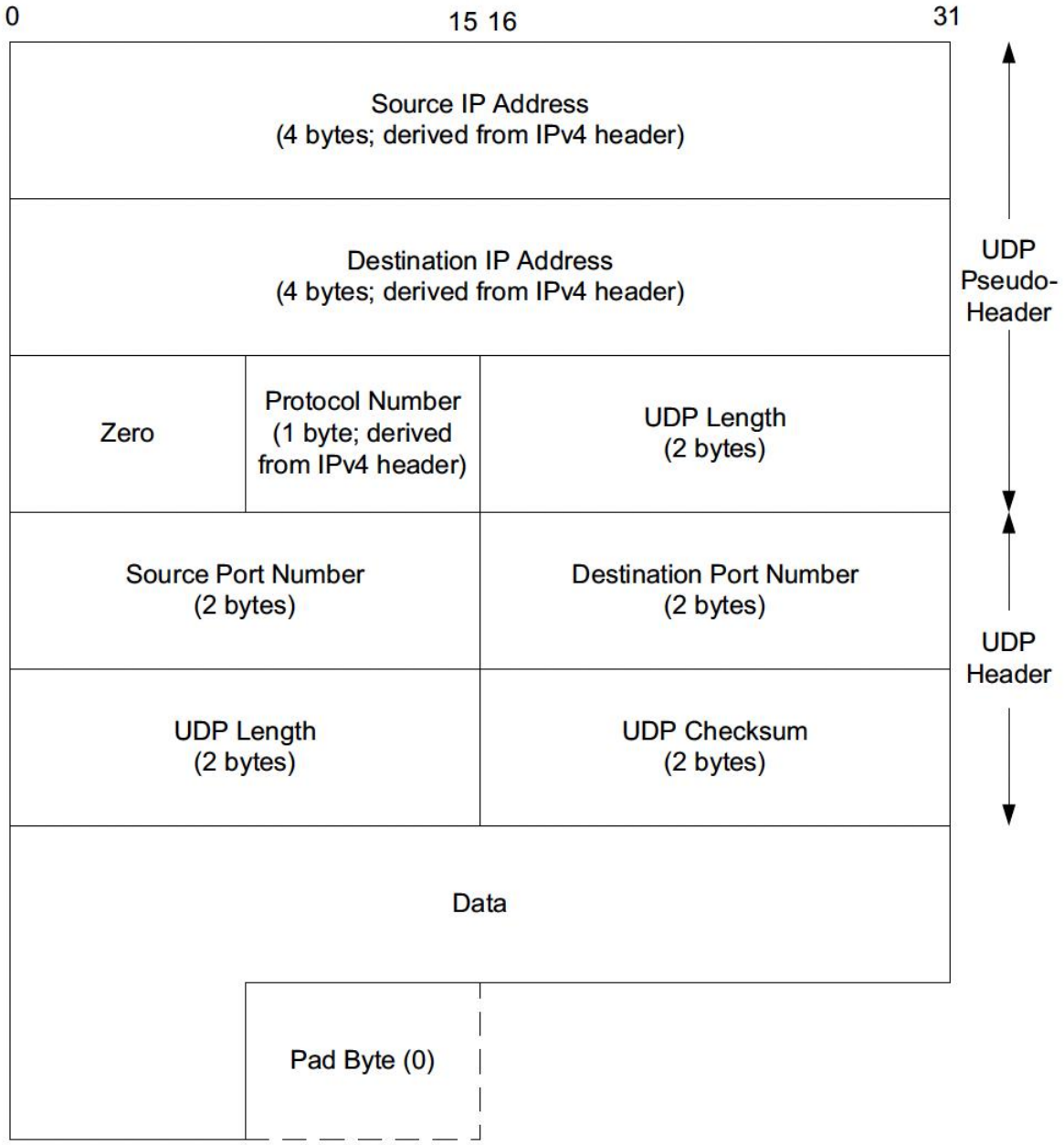
- treat contents of UDP segment and a pseudo-header (including IP addresses) as a sequence of 16-bit integers
- checksum: add (1's complement sum) those integers, and invert all the bits of the sum
- checksum value put into UDP checksum field

receiver:

- compute checksum of received segment
- check if computed checksum equals checksum field value:
  - not equal - error detected
  - equal - no error detected. (But may actually have errors - see slide 7-34)

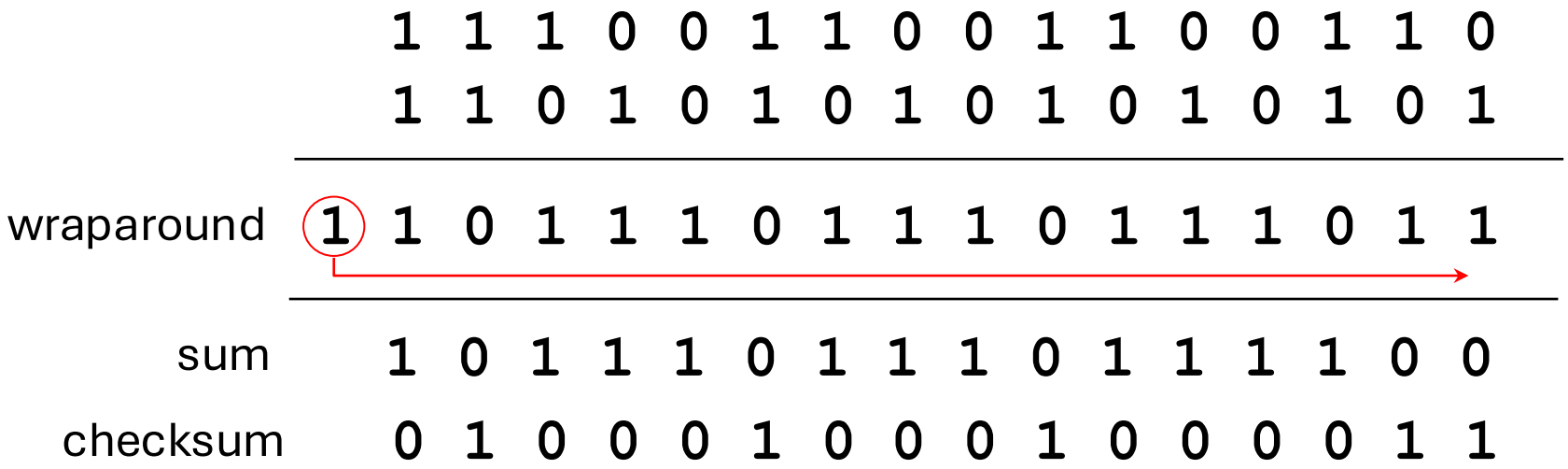
UDP checksum (ctd)

- For calculating checksum, UDP includes a pseudo-header made up from fields in the IP datagram
- A pseudo-header similar to this is used the calculating the TCP checksum as well
- Pseudo-header is an imaginary header used for checksum calculation only
- It is never transmitted



Internet checksum: an example

example: add two 16-bit integers

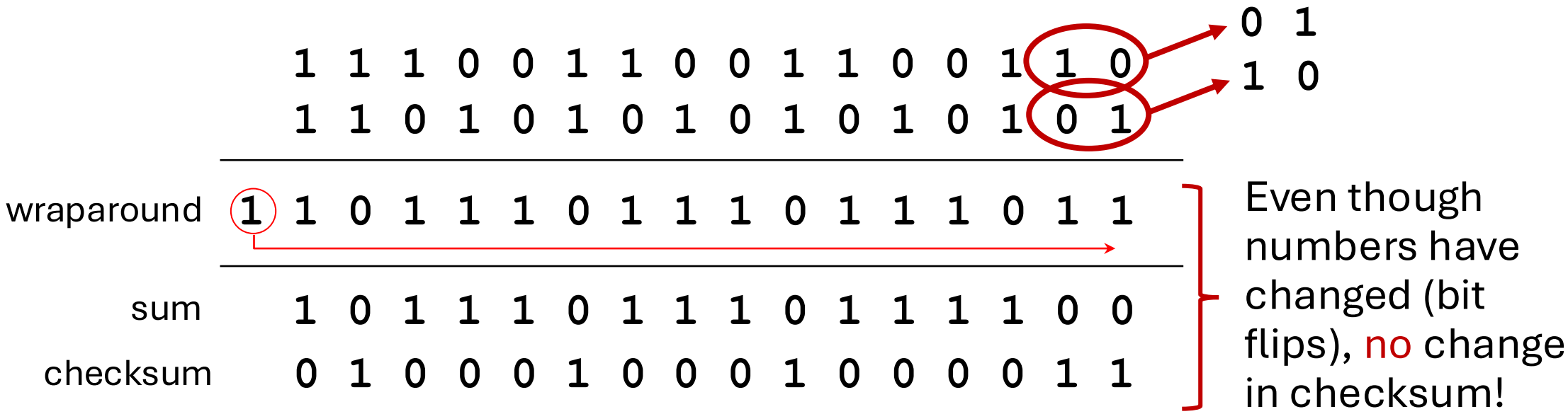


Note: when adding numbers, a carryout from the most significant bit needs to be added to the result

* Check out the online interactive exercises for more examples: http://gaia.cs.umass.edu/kurose_ross/interactive/

Internet checksum: weak protection!

example: add two 16-bit integers



Summary: UDP

- "no frills" (simple) protocol:
  - segments may be lost, delivered out of order
  - best effort service: "send and hope for the best"
- UDP has its plusses:
  - no setup/handshaking needed (no RTT incurred)
  - can function when network service is compromised
  - helps with reliability (checksum)
- build additional functionality on top of UDP in application layer (e.g., HTTP/3)

Lesson outline

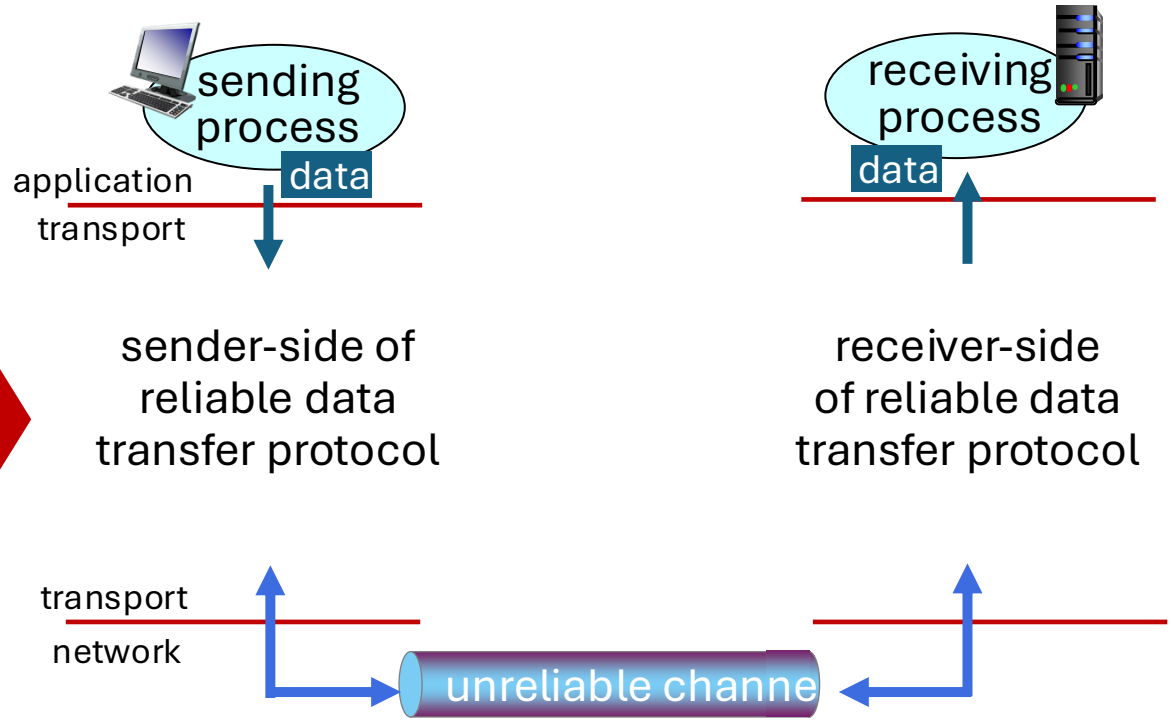
- Transport-layer services
- Multiplexing and demultiplexing
- Connectionless transport: UDP
- Connection-oriented transport: TCP
- TCP connection management
- TCP Sequence numbers and acknowledgements
- TCP Retransmission
- TCP Flow control
- TCP Congestion control

Principles of reliable data transfer



reliable service abstraction:
Sending process thinks that the underlying channel is reliable

reliable service implementation:
Transport layer does additional work to provide a reliable service over the unreliable service provided by IP layer



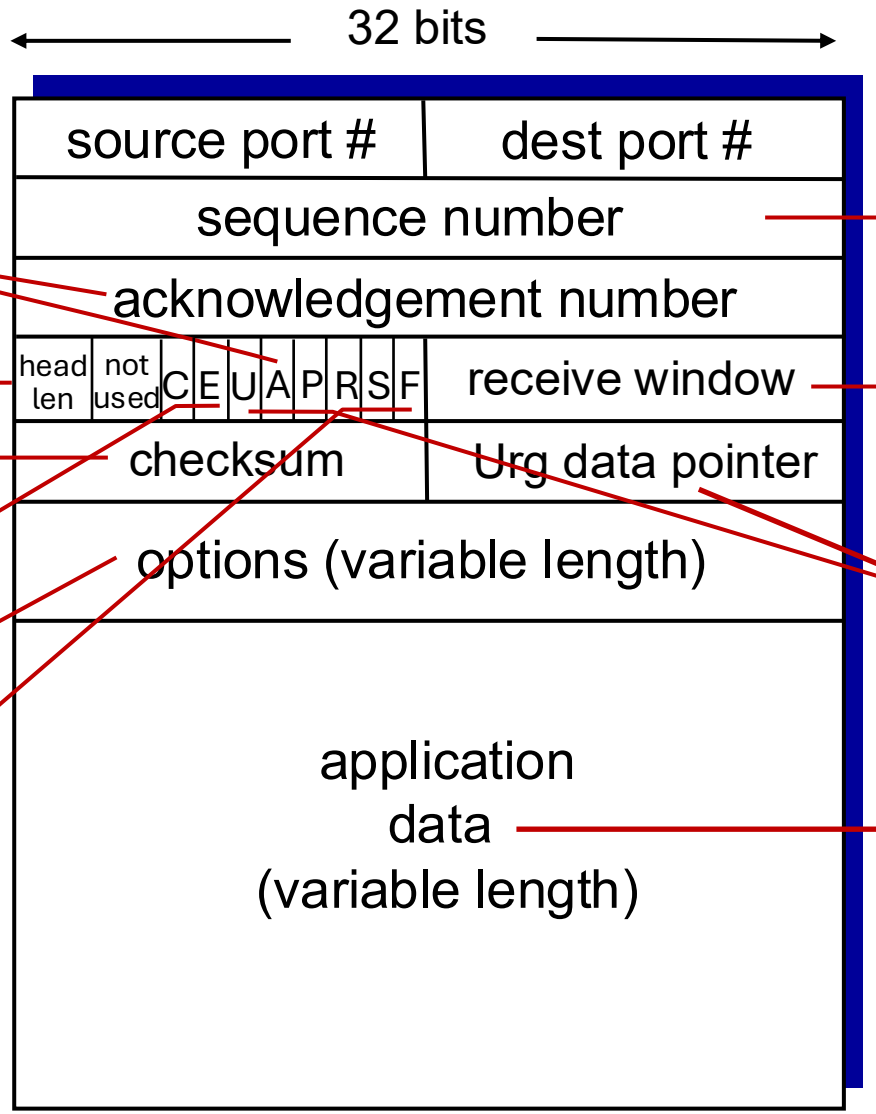
TCP: overview RFCs: 793,1122, 2018, 5681, 7323

- point-to-point:
  - one sender, one receiver
- connection-oriented:
  - connection established before sending any data
  - connection closed after data transfer is over
- full duplex data:
  - bi-directional data flow in same connection
  - MSS: maximum segment size each host willing to receive specified during connection establishment
- reliable, in-order byte stream:
  - each byte in each stream has a unique sequence no.
- cumulative ACKs:
  - sequence number of last byte received is acknowledged by receiver
- retransmission:
  - sequence number of last byte received is acknowledged by receiver
- flow control:
  - sender will not overwhelm receiver
- congestion control:
  - If sender observes congestion, it will lower the sending rate

TCP: overview RFCs: 793,1122, 2018, 5681, 7323

- point-to-point:
  - one sender, one receiver
- connection-oriented:
  - connection established before sending any data
  - connection closed after data transfer is over
- full duplex data:
  - bi-directional data flow in same connection
  - MSS: maximum segment size each host willing to receive specified during connection establishment
- reliable, in-order byte stream:
  - each byte in each stream has a unique sequence no.
- cumulative ACKs:
  - sequence number of last byte received is acknowledged by receiver
- retransmission:
  - If acknowledgement for segment received within a certain time, sender will retransmit
- flow control:
  - Receiver can ask sender to slow down
- congestion control:
  - If sender observes congestion, it will lower the sending rate

TCP segment structure



ACK: seq # of next byte expected;
A bit: this is an ACK

length (of TCP header)

Internet checksum

C, E: congestion notification

TCP options

RST, SYN, FIN: connection management

segment seq #: counting bytes of data into bytestream (not segments!)

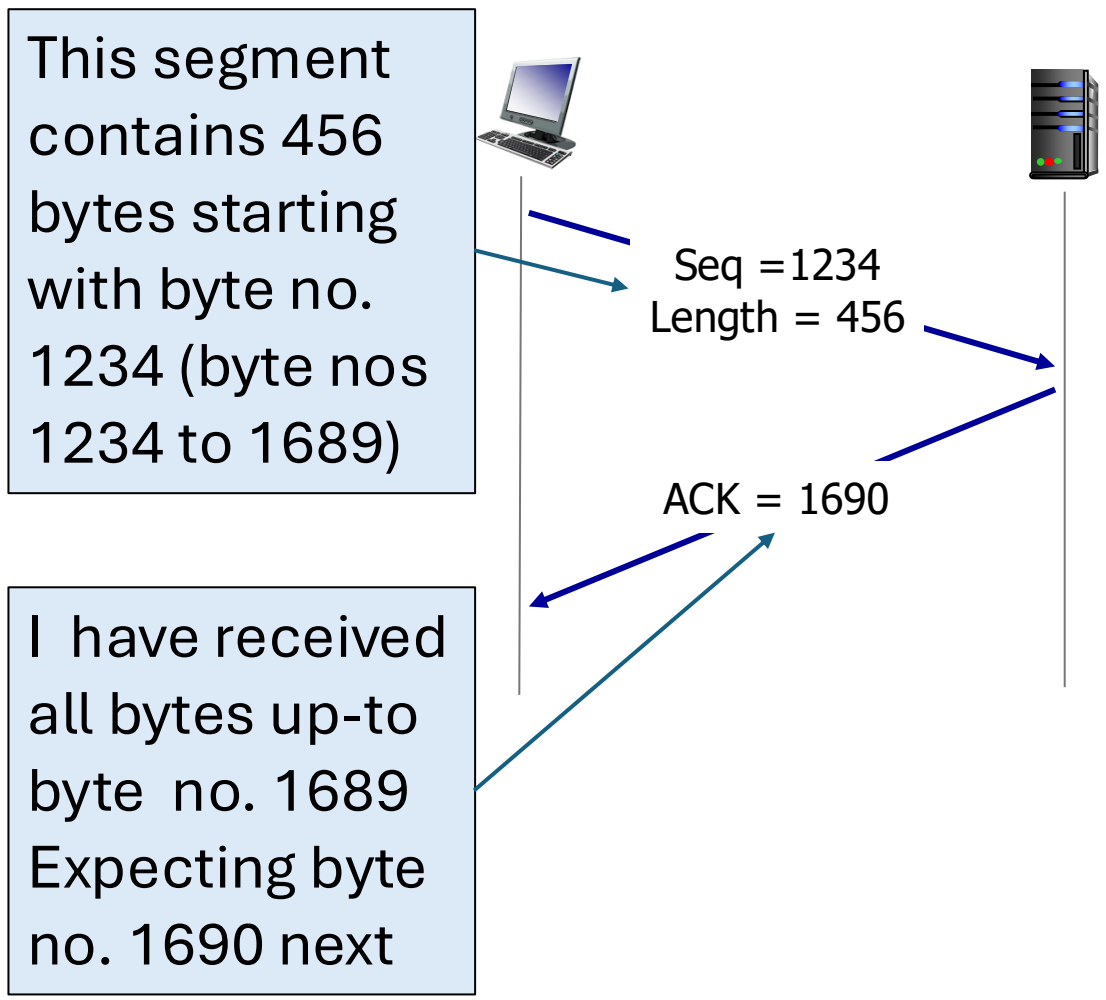
flow control: # bytes receiver willing to accept

location of data to be delivered urgently

data sent by application into TCP socket

Sequence numbers and acknowledgements

- Every byte in data sent in a connection is identified by a sequence number
- Every segment contains the sequence number of the first byte in the segment
- When a segment is received by destination host, it acknowledges the last sequence number received



Lesson outline

- Transport-layer services
- Multiplexing and demultiplexing
- Connectionless transport: UDP
- Connection-oriented transport: TCP
- TCP connection management
- TCP Sequence numbers and acknowledgements
- TCP Retransmission
- TCP Flow control
- TCP Congestion control

TCP connection establishment

- Before exchanging data, sender/receiver need to establish a connection
- Server must be “listening” on a particular port, expecting connections from clients
- Client initiates the connection establishment
  - Create a socket on the client host
  - “Connect” the socket to the Server-IP : Port
- Connection establishment requires the “three-way-handshake”
  - Server must accept the connection request send by the client
  - Both agree on connection parameters (e.g., starting sequence numbers, receive buffer sizes)

TCP 3-way handshake

Notation:
ACK=79 means ACK flag (A) is 1 and Acknowlegment number =79
ACK=0 means ACK flag (A) is 0

Client state

```
clientSocket = socket(AF_INET, SOCK_STREAM)
```

CLOSED

```
clientSocket.connect((serverName,serverPort))
```



choose init seq num, x
send TCP SYN msg

SYN=1, Seq=x
ACK=0

SYNSENT



choose init seq num, y
send TCP SYNACK
msg, acking SYN

SYN=1, Seq=y
ACK=x+1

received SYNACK(x)
indicates server is live;
send ACK for SYNACK;
this segment may contain
client-to-server data

Seq=x+1,
ACK=y+1

received ACK(y)
indicates client is live

ESTAB

Server state

```
serverSocket = socket(AF_INET,SOCK_STREAM)
serverSocket.bind(('',serverPort))
serverSocket.listen(1)
connectionSocket, addr = serverSocket.accept()
```

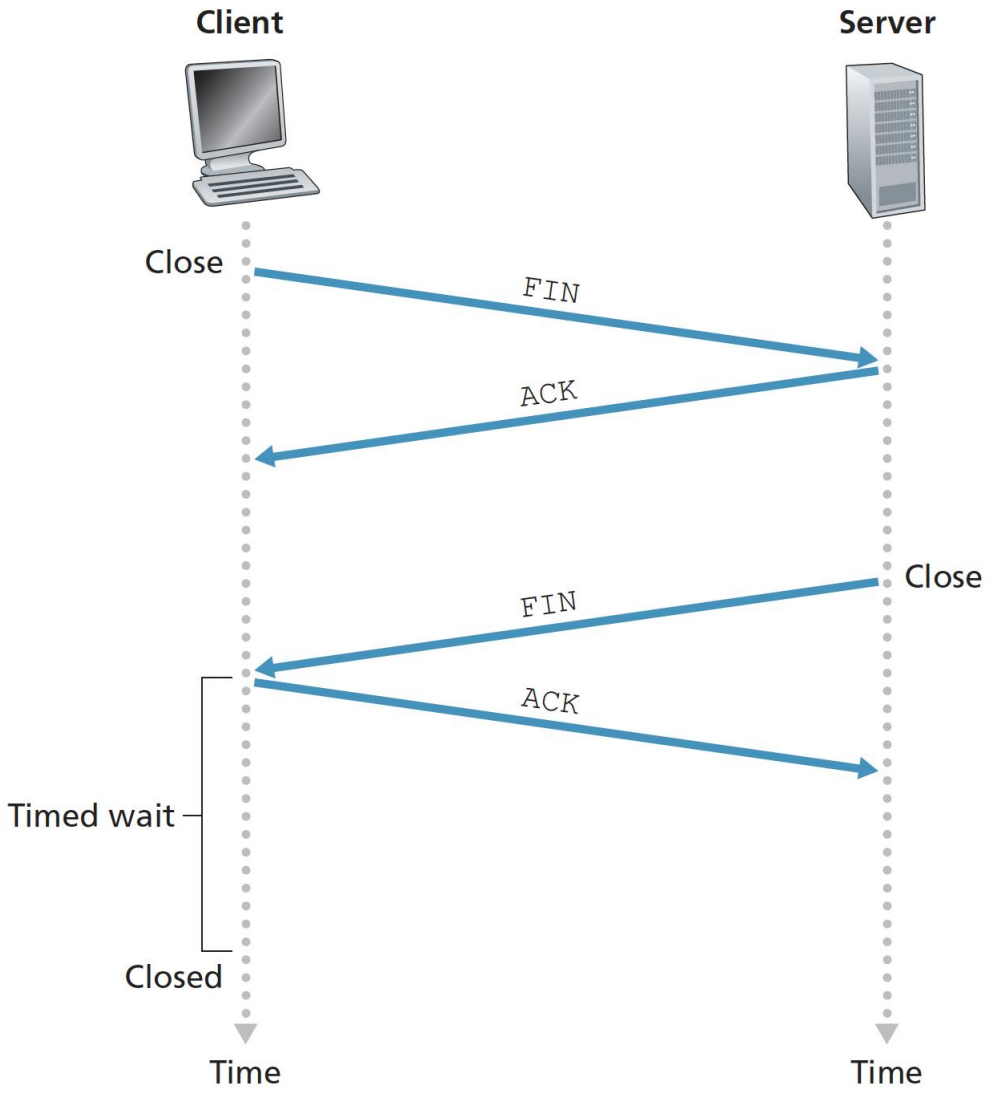
LISTEN

SYN RCVD

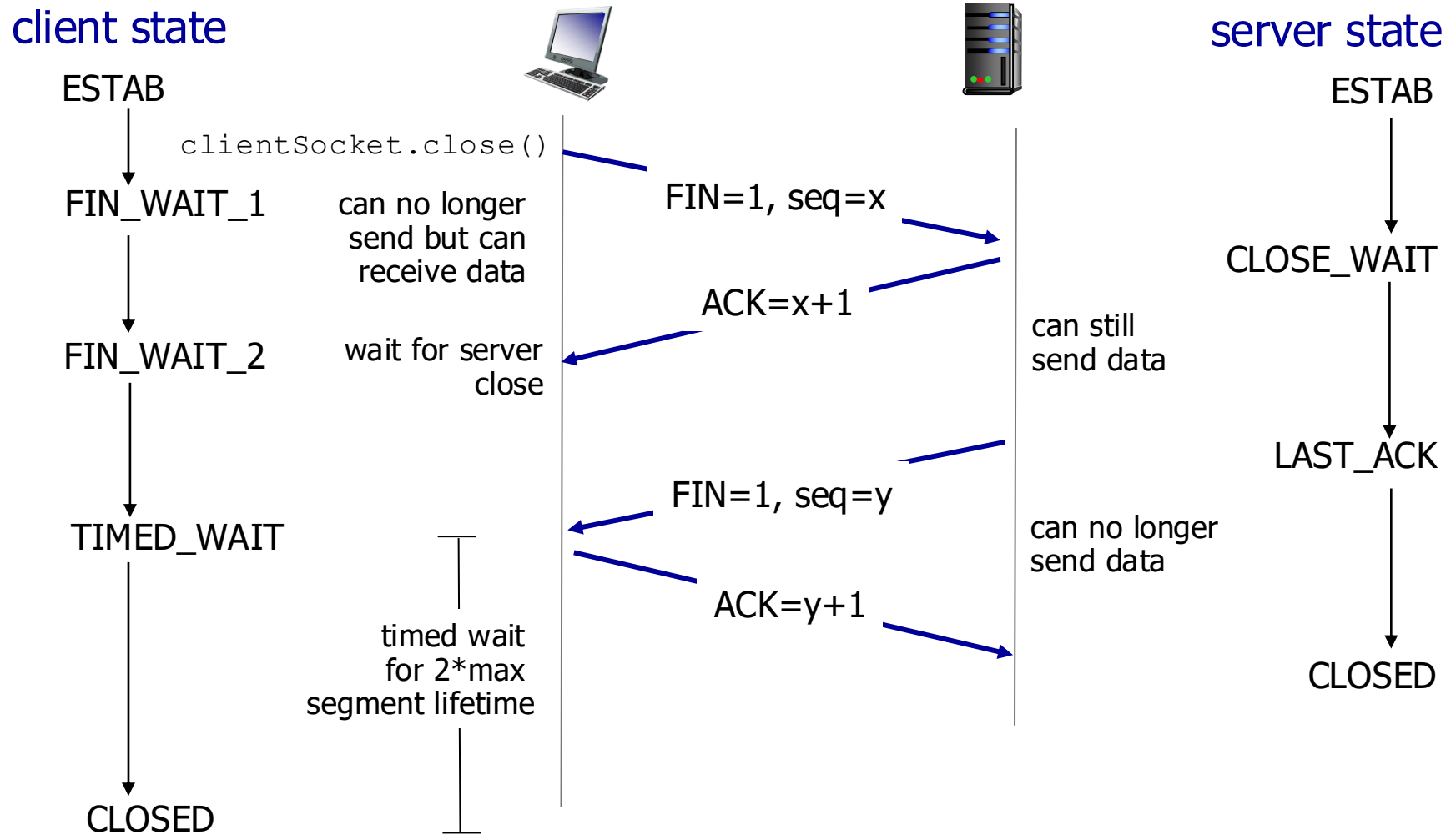
ESTAB

Closing a TCP connection

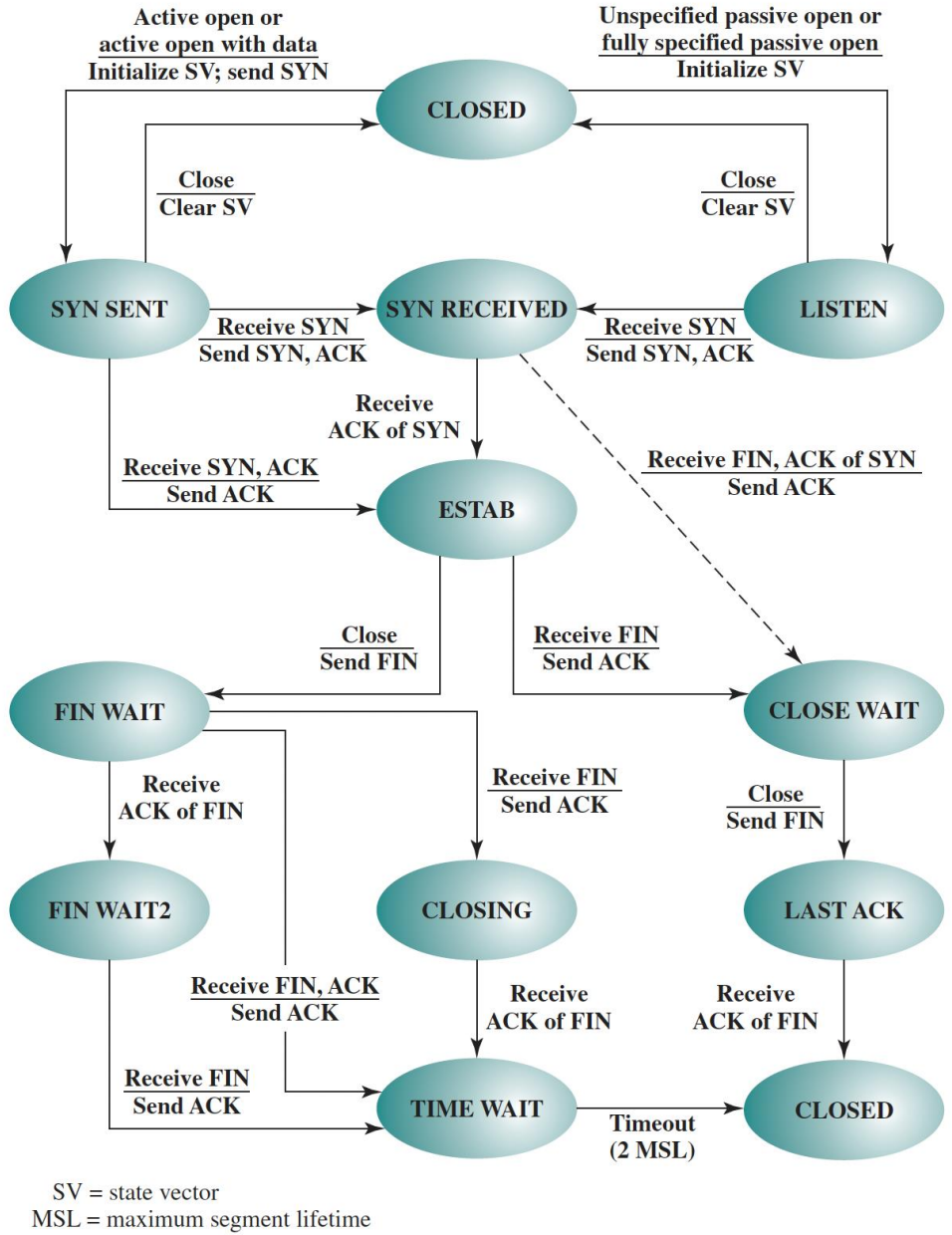
- client, server each close their side of connection
  - send TCP segment with FIN bit = 1
- respond to received FIN with ACK
  - on receiving FIN, ACK can be combined with own FIN
- simultaneous FIN exchanges can be handled



Closing a TCP connection (2)

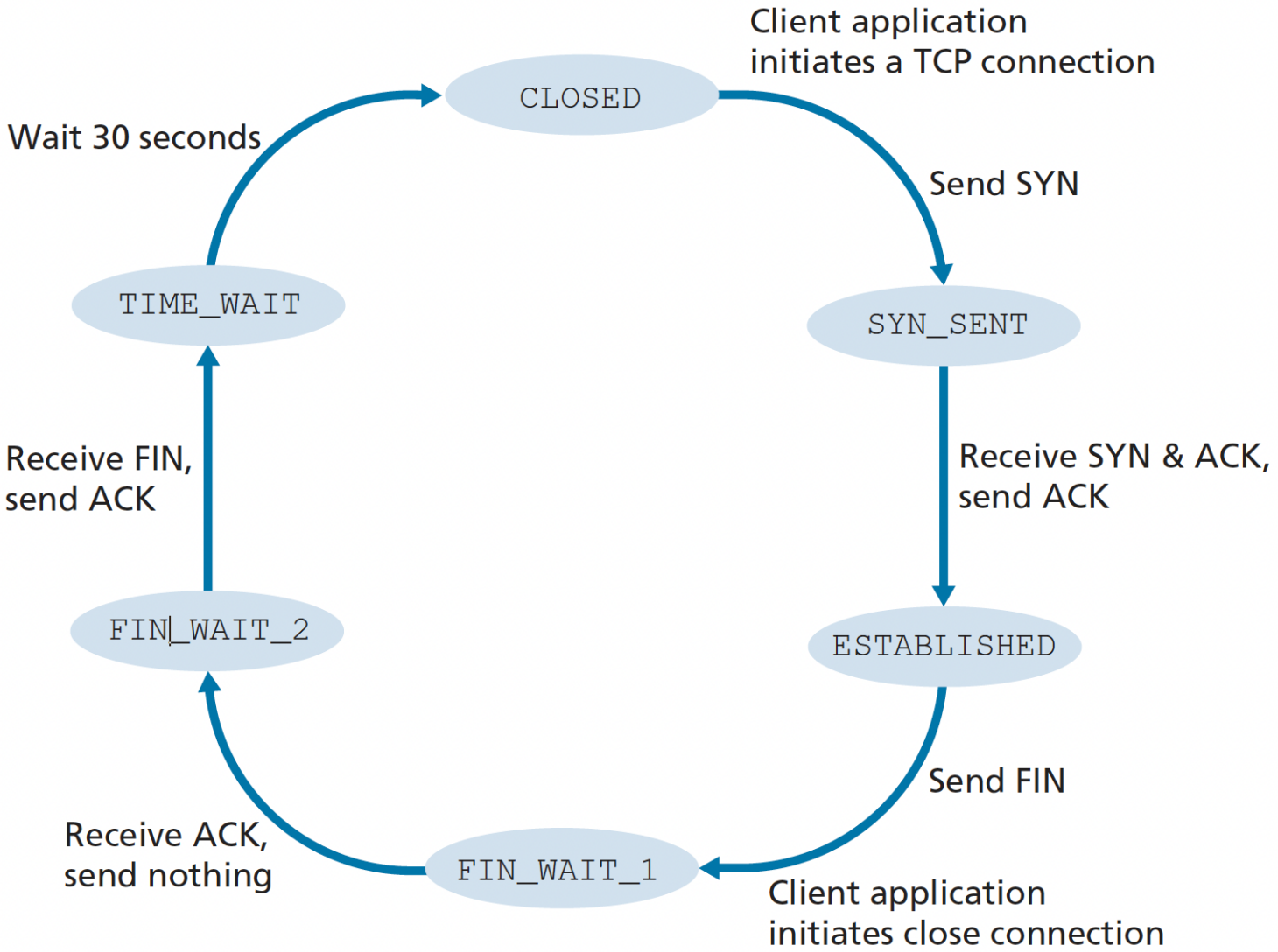


TCP State Diagram



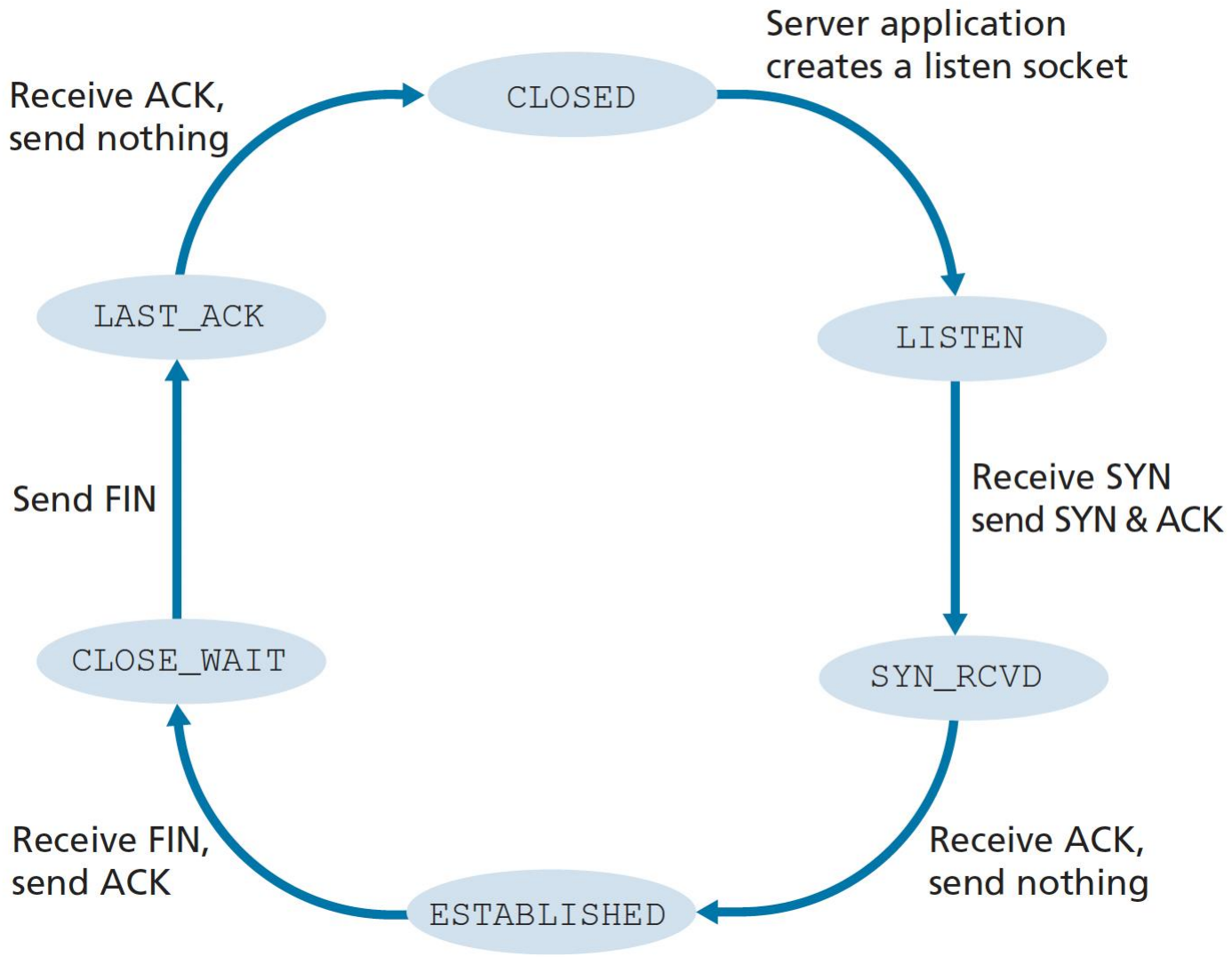
TCP State sequence (Client)

- Typical sequence of states visited by a client



TCP State sequence (Server)

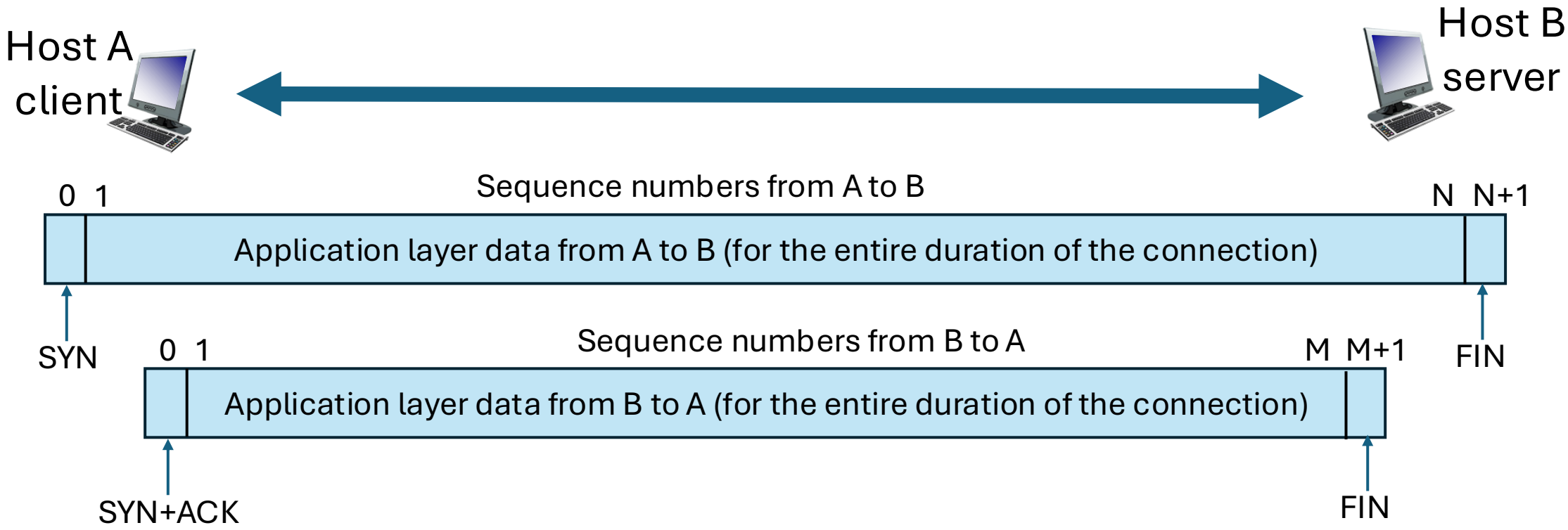
- Typical sequence of states visited by a server



Lesson outline

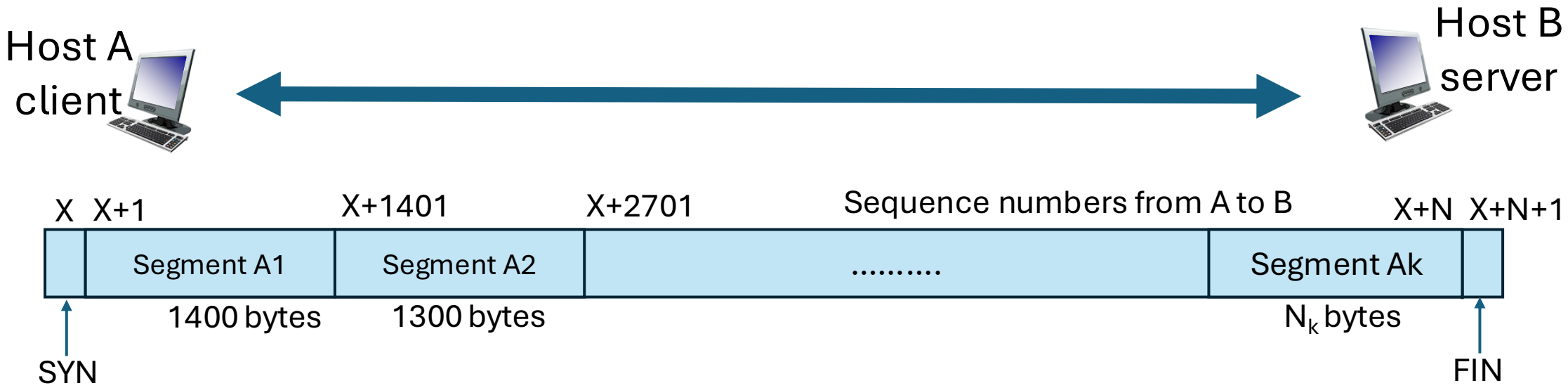
- Transport-layer services
- Multiplexing and demultiplexing
- Connectionless transport: UDP
- Connection-oriented transport: TCP
- TCP connection management
- TCP Sequence numbers and acknowledgements
- TCP Retransmission
- TCP Flow control
- TCP Congestion control

TCP sequence numbers



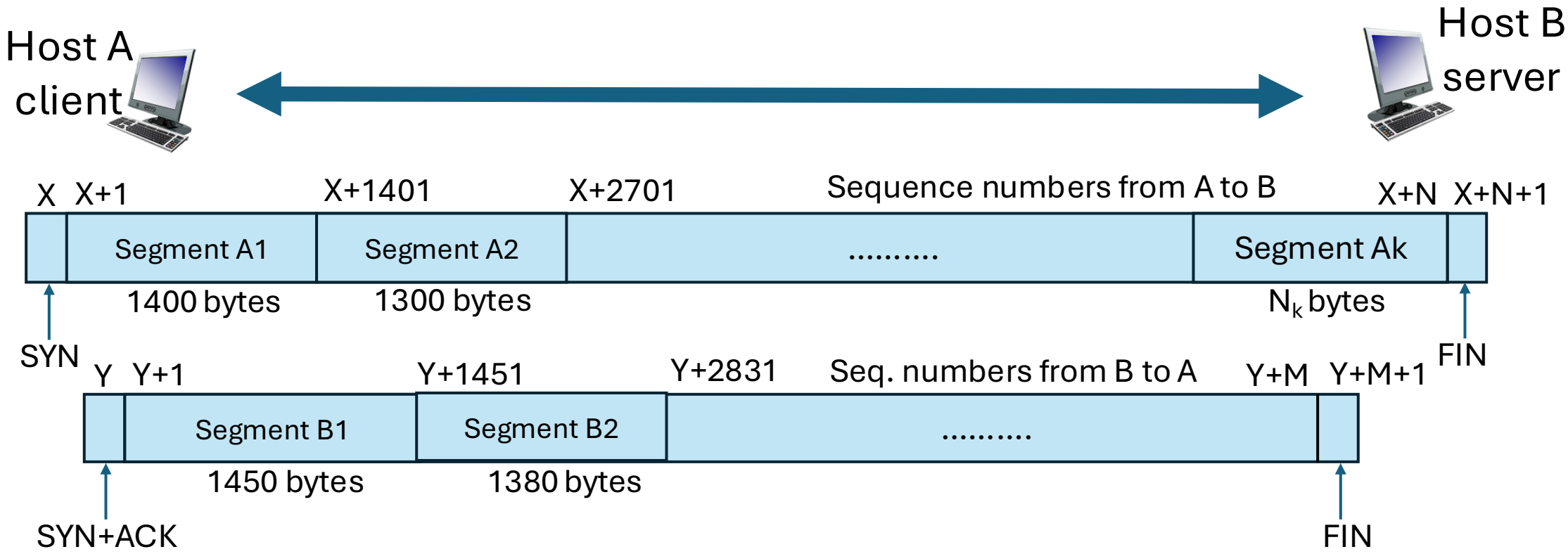
- In practice, sequence numbers in each direction start at a random number instead of 0
- SYN, SYN+ACK and FIN segments are considered to have one byte of data although in reality they do not contain any data.

TCP sequence numbers and ACKs



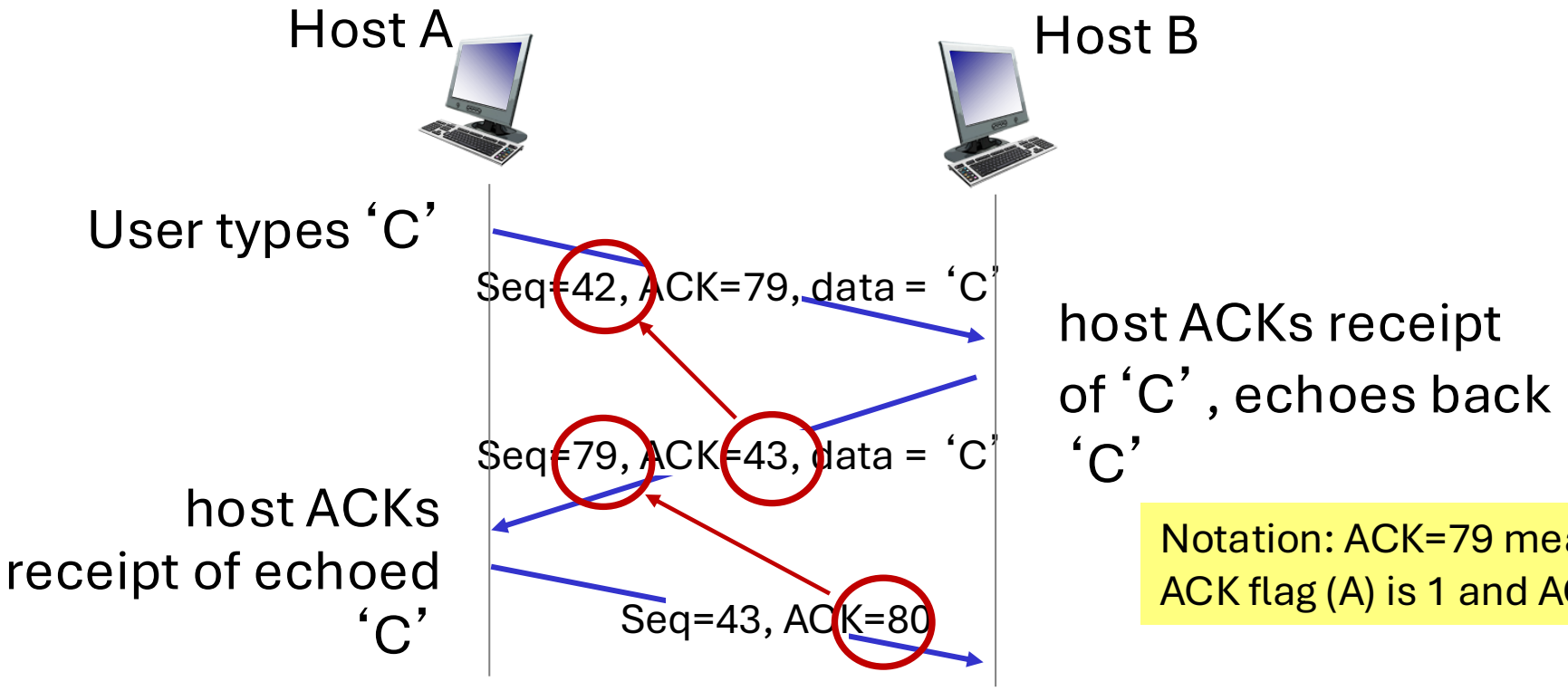
- Let's say that data from A to B is sent in k segments (Segment numbers 1,2,...k)
- Size (no. of bytes) of segments are N_1, N_2,..., N_k and $N = N_1+N_2+...+N_k$
- After receiving a segment, the recipient is supposed to acknowledge (ACK) with Acknowledgement number = sequence number of next byte expected
  - After receiving segment A1, B can respond with ACK = $X+1401$
  - After receiving segments A1 and A2, B can respond with ACK = $X+2701$
- Acknowledgements are cumulative
  - Instead of sending two ACKs as above, B can send only one ACK with ACK = $X+2701$ after receiving segment A2

TCP sequence numbers and ACKs



- Sending an ACK segment with no data is wasteful
- Recipient can combine an ACK with one of its own data segments
  - B can combine ACK for A's segment A1 with its own segment B2
  - Segment B2 can have Seq. no = $Y+1451$ and ACK = $X+1401$

TCP sequence numbers, ACKs

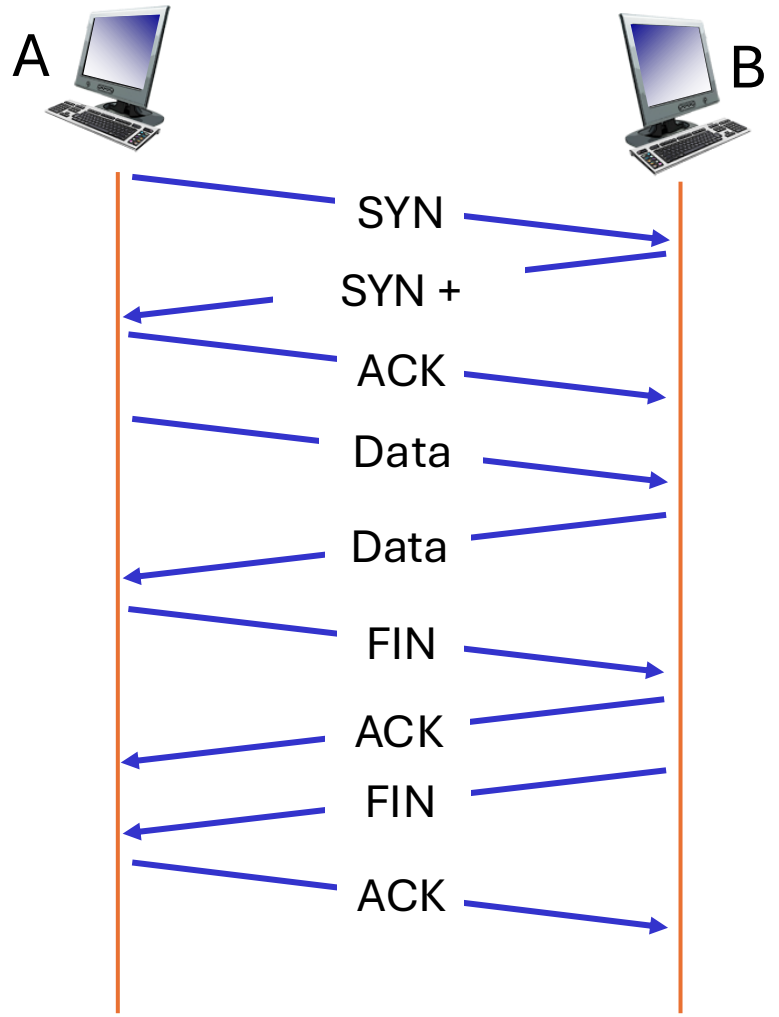


Notation: ACK=79 means that the ACK flag (A) is 1 and ACKnum=79

simple telnet scenario

Example: TCP client and server

- Client host A is communicating with server B listening on TCP port 2000.
- Client opens the TCP connection to the server, and then sends the text message `Hello!`
- When the message is received by server, it responds with the text message `Welcome!`
- Upon receiving the text message from server, client closes the TCP connection
- Finally, server also closes the connection



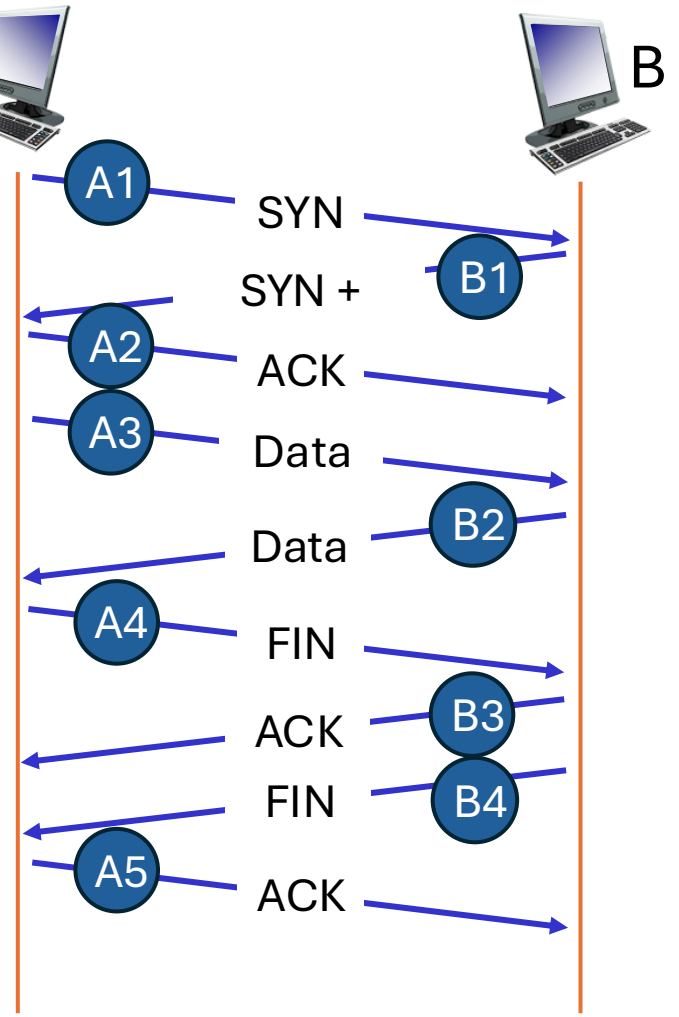
Example: TCP client and server (2)

Let's say A's Seq. No. starts at 10000 and B's Seq. No. starts at 20000

- A1: Seq=10000, SYN=1, ACK=0
- B1: Seq=20000, SYN=1, ACK=10001
- A2: Seq=10001, ACK=20001
- A3: Seq=10001, ACK=20001, Len=6, Data="Hello!"
- B2: Seq=20001, ACK=10007, Len=8, Data="Welcome!"
- A4: Seq=10007, ACK=20009, FIN=1
- B3: Seq=20009, ACK=10008
- B4: Seq=20009, ACK=10008, FIN=1
- A5: Seq=10008, ACK=20010

All segments except A3 and B2 have no data (i.e. Len=0)

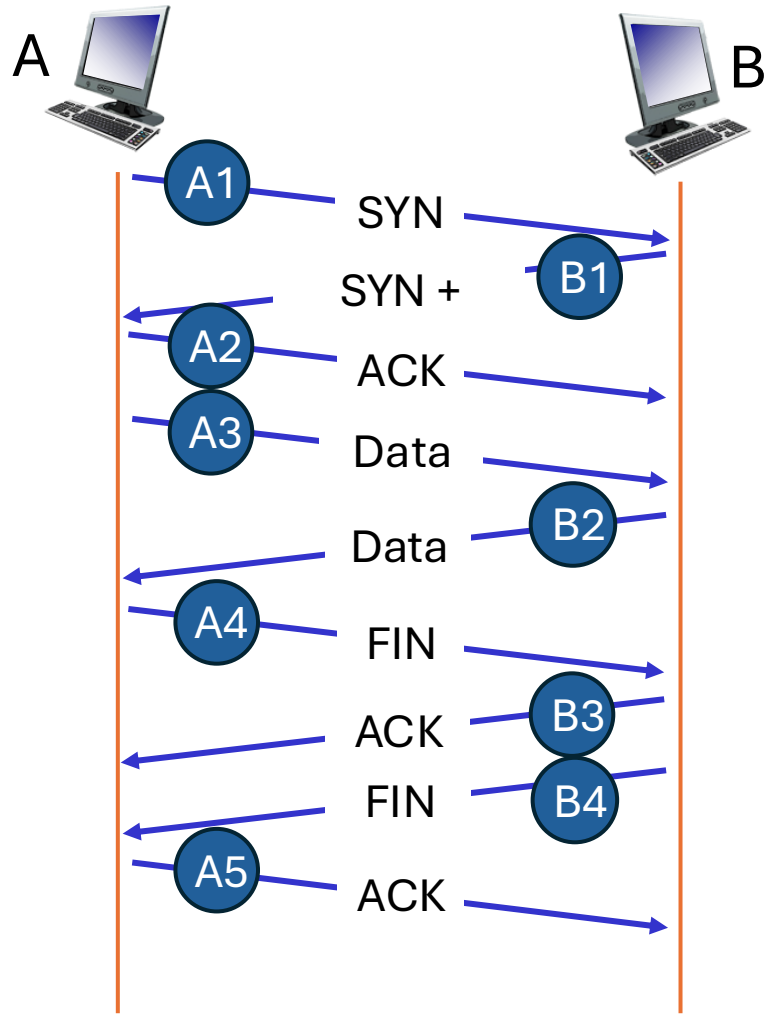
SYN, SYN+ACK, and FIN Segments are considered to have a length of 1 byte, although they have no data (Len=0)



Exercise: TCP client and server

Exercise:
Two of the segments shown here (one from A and one from B) can be combined with the next segment from the same host, to reduce the total number of segments in the connection. Can you identify the two segments?

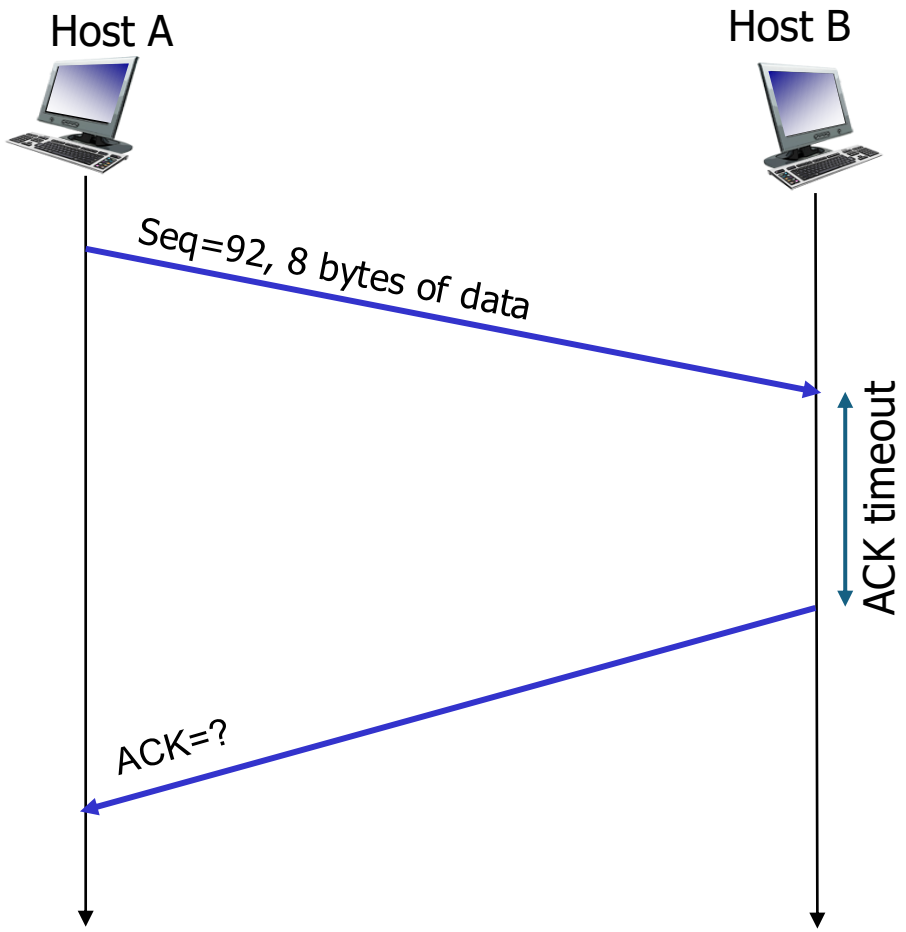
Hint: Check all the segments with no data.

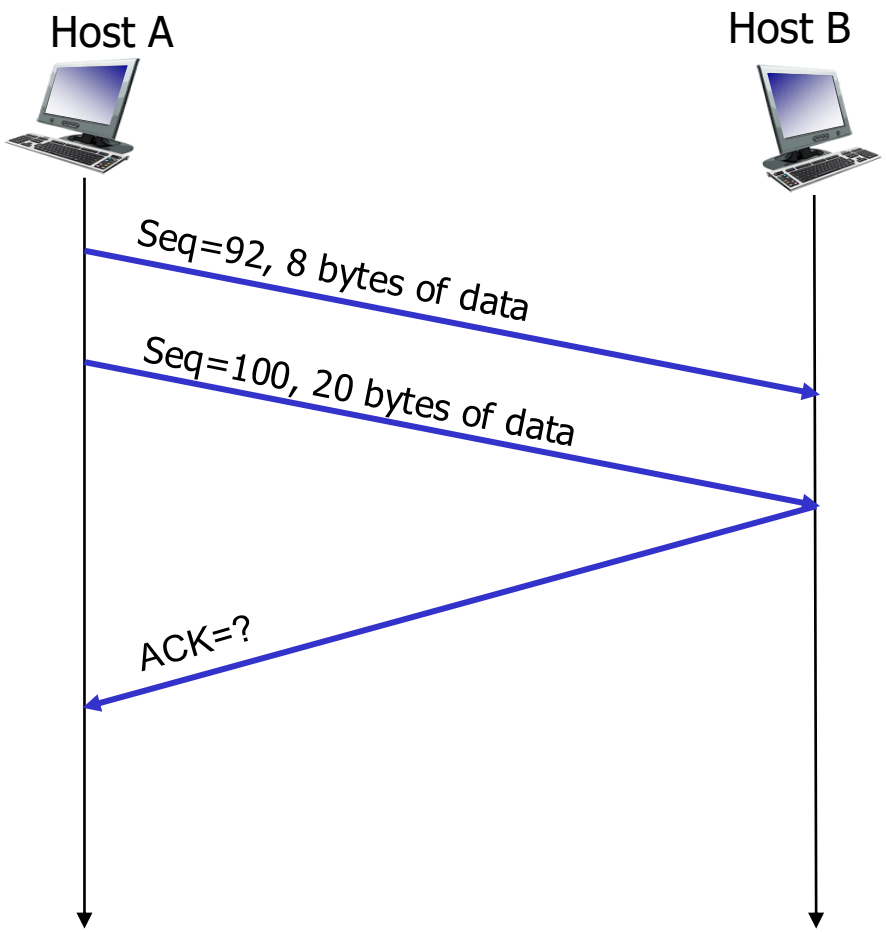


TCP Receiver: ACK generation [RFC 5681]

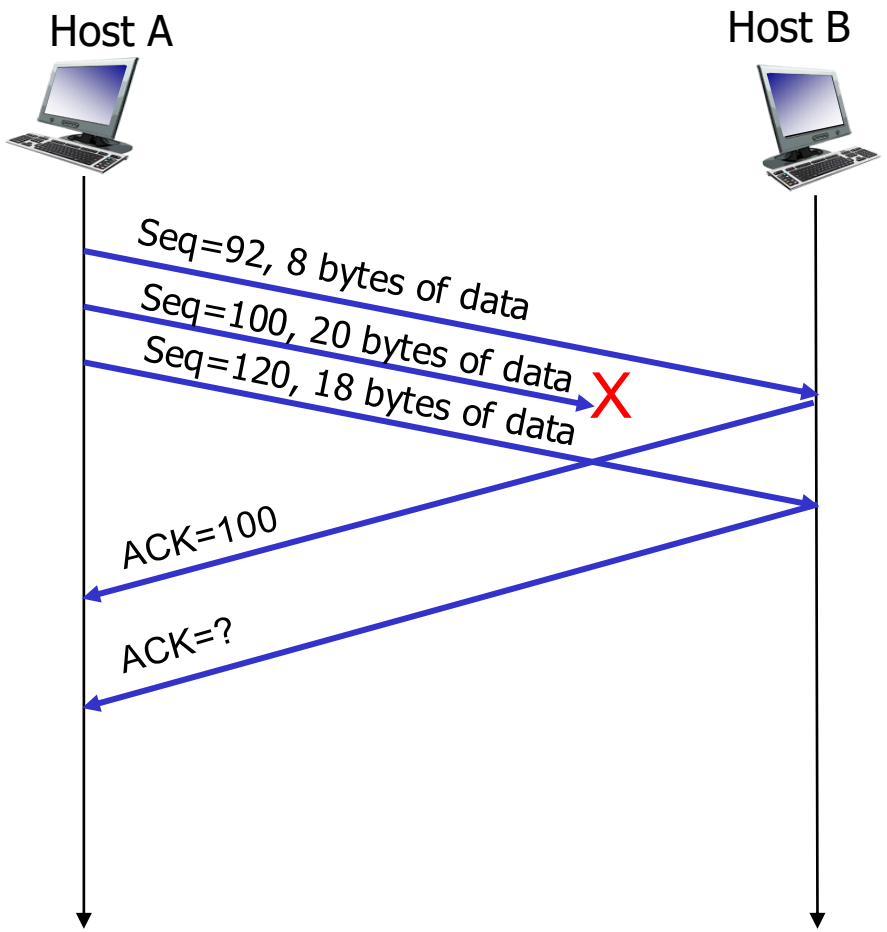
Event at receiver	TCP receiver action
arrival of in-order segment with expected seq. no. All data up to expected seq. no. already ACKed	delayed ACK. Wait up to 500ms for next segment. If no next segment within that time, send ACK
arrival of in-order segment with expected seq. no. when one other segment has ACK pending	immediately send single cumulative ACK, ACKing both in-order segments
arrival of out-of-order segment higher-than-expected seq. no. gap detected	immediately send duplicate ACK, indicating seq. # of next expected byte
arrival of segment that partially or completely fills gap	immediately send ACK, provided that segment starts at lower end of gap

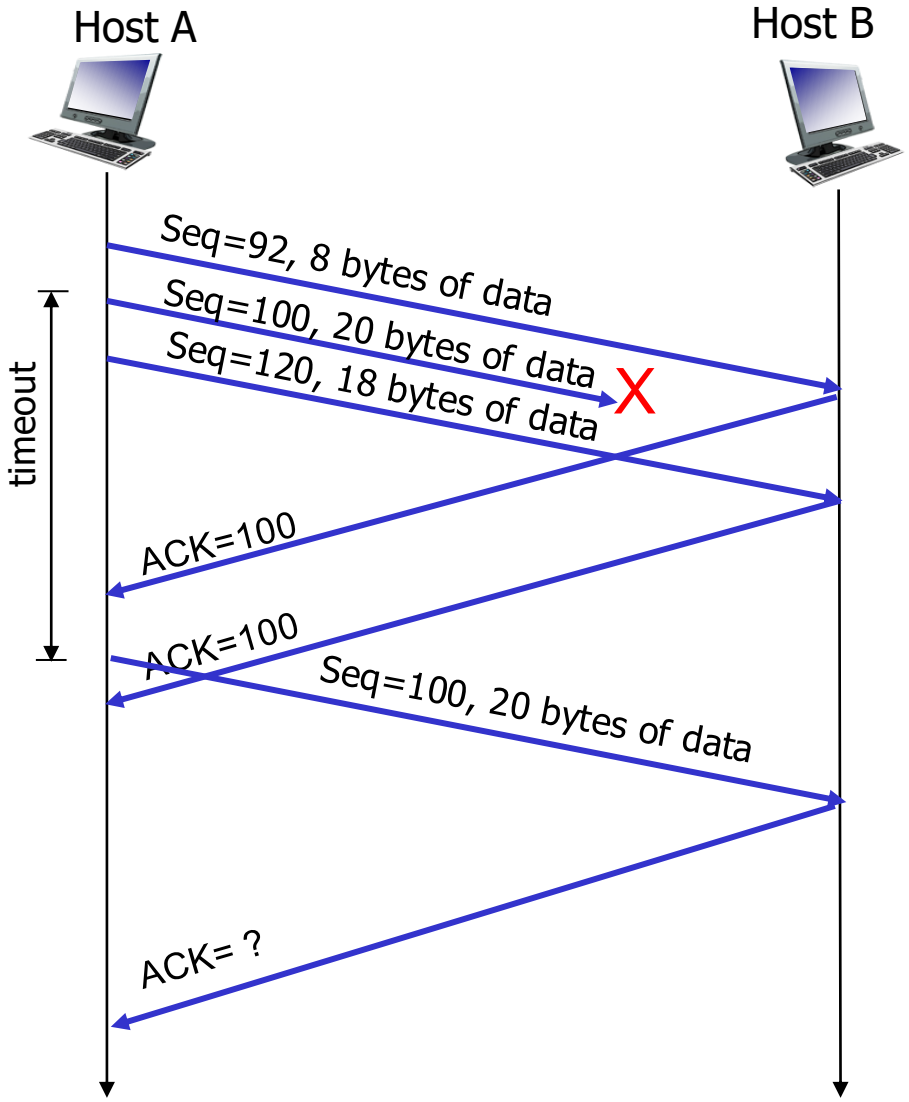
ACK generation exercises





ACK generation exercises





Review questions

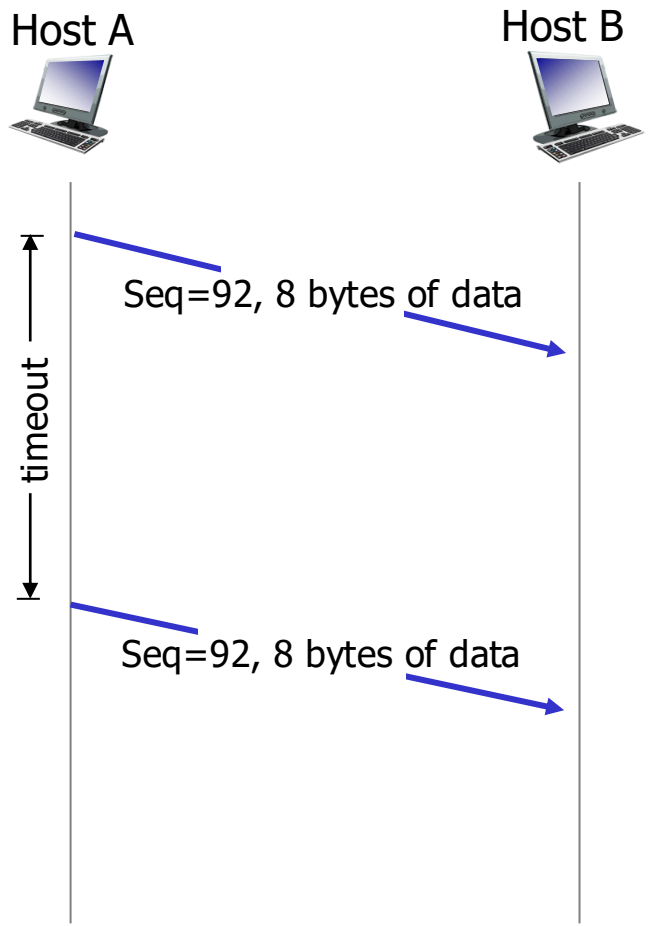
- What are the similarities and differences in UDP and TCP sockets?
- What are the parameters required to fully describe a
  - UDP socket
  - TCP socket
- Show segments exchanged between a client and a server doing the following transaction in a timing diagram (like slides 60-62)
  - Client opens a TCP connection to server
  - Client sends 200 bytes of data
  - Server sends 400 bytes of data
  - Client closes connection
  - Server closes connection
- Determine the sequence number, acknowledgement number, data portion size, and flags that would be set in each segment in the above transaction. Assume initial sequence numbers of 2000 and 4000 respectively for the client and server.

Lesson outline

- Transport-layer services
- Multiplexing and demultiplexing
- Connectionless transport: UDP
- Connection-oriented transport: TCP
- TCP connection management
- TCP Sequence numbers and acknowledgements
- TCP Retransmission
- TCP Flow control
- TCP congestion control

TCP retransmission

- If an ACK is not received within a certain time period after sending a segment, TCP will retransmit the segment
- This is called the Timeout period



TCP Sender (simplified)

event: data received from application

- create segment with seq no.
- seq no. is byte-stream number of first data byte in segment
- start timer if not already running
  - timer is for the oldest unACKed segment
  - called RTO (Retransmission Time Out)

event: timeout

- retransmit segment that caused timeout
- restart timer

event: ACK received

- if ACK acknowledges previously unACKed segments
  - update what is known to be ACKed
  - set RTO for the next unACKed segment (if any)

TCP round trip time (RTT), timeout

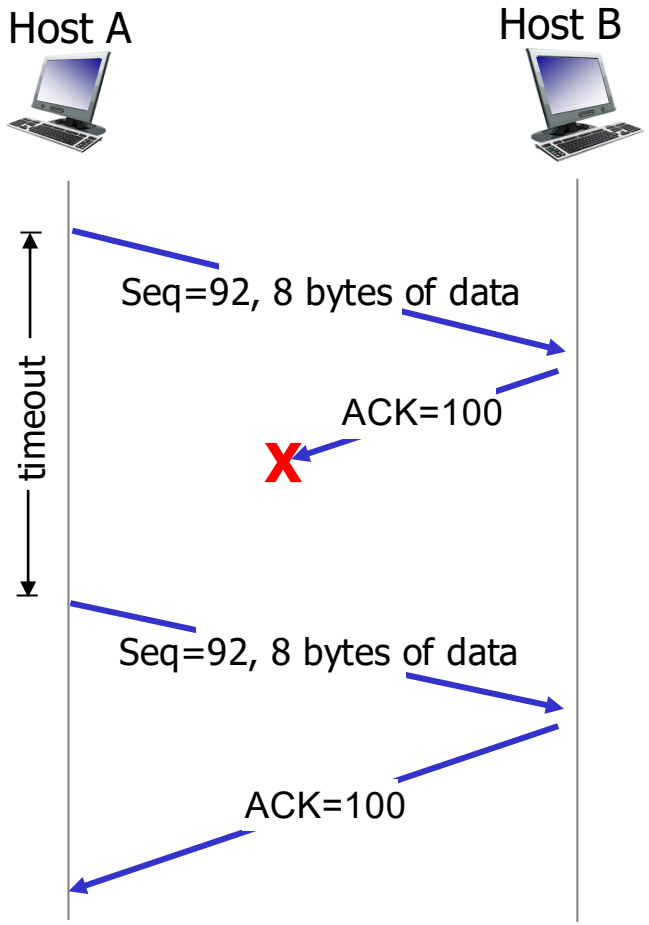
Q: how to set TCP timeout value?

- longer than RTT, but RTT varies!
- too short: premature timeout, unnecessary retransmissions
- too long: slow reaction to segment loss

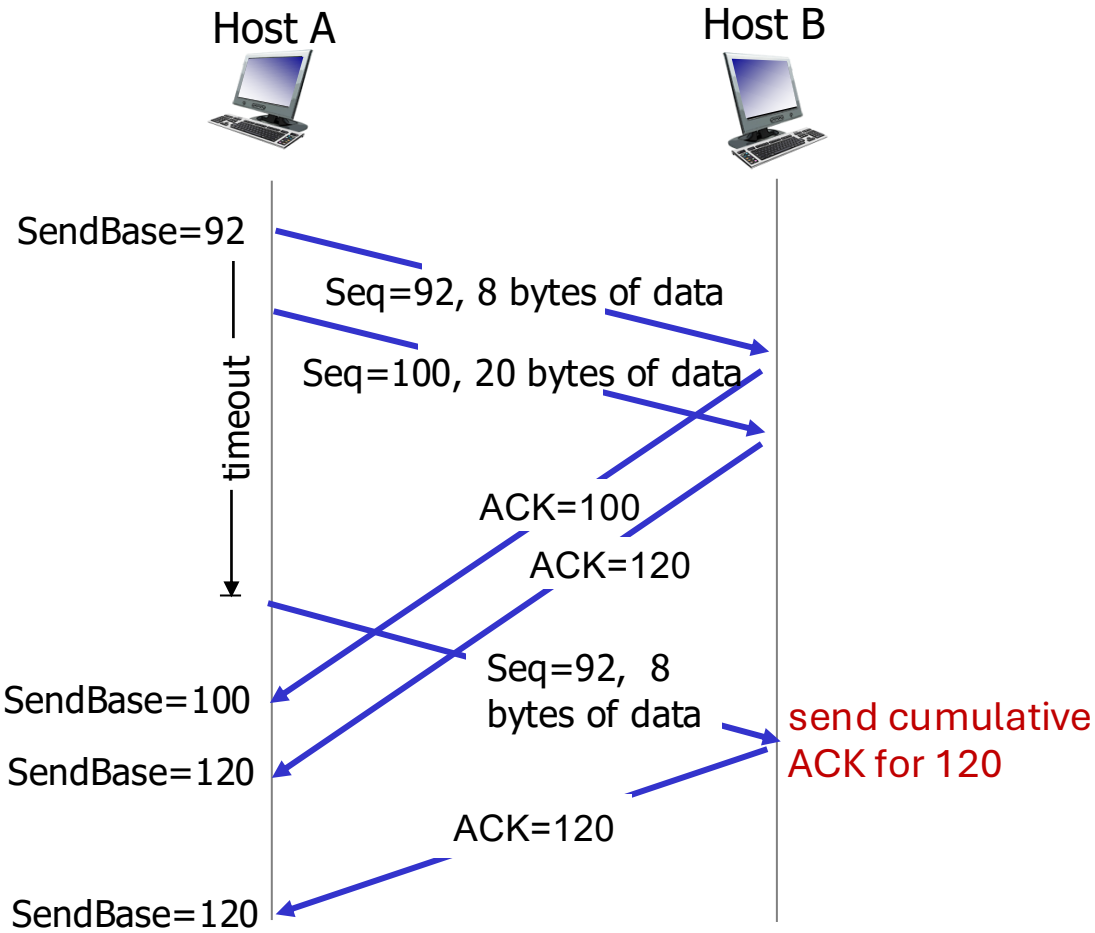
Q: how to estimate RTT?

- `SampleRTT` = measured time from segment transmission until ACK receipt
  - ignore retransmissions
- `SampleRTT` will vary from segment to segment
  - average several recent measurements, not just the last `SampleRTT`

TCP: retransmission scenarios

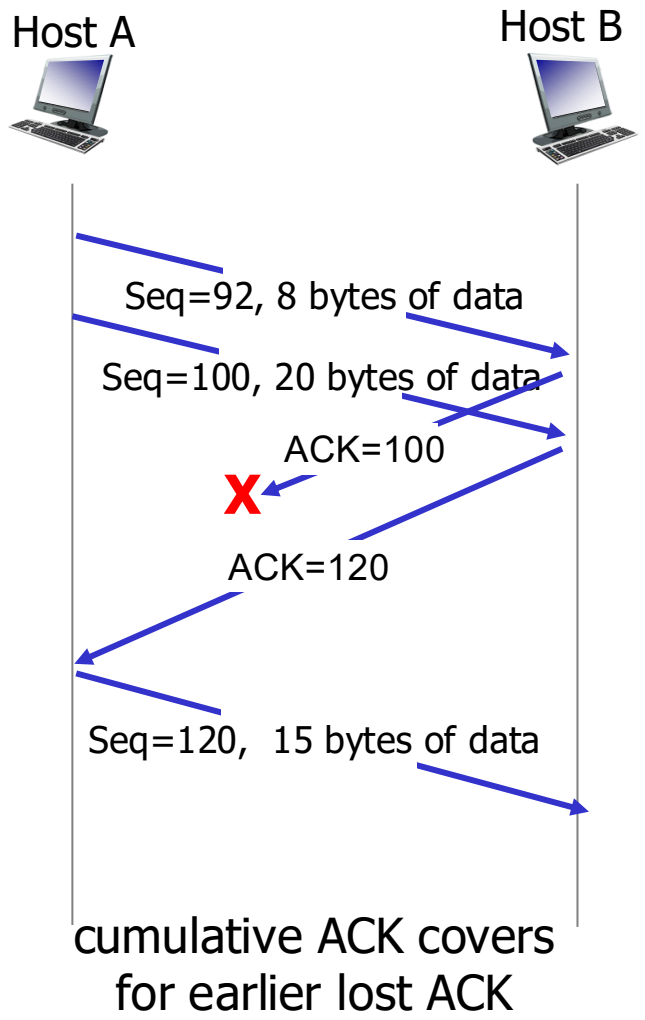


lost ACK scenario



premature timeout RTO < RTT

TCP: retransmission scenarios



TCP fast retransmit

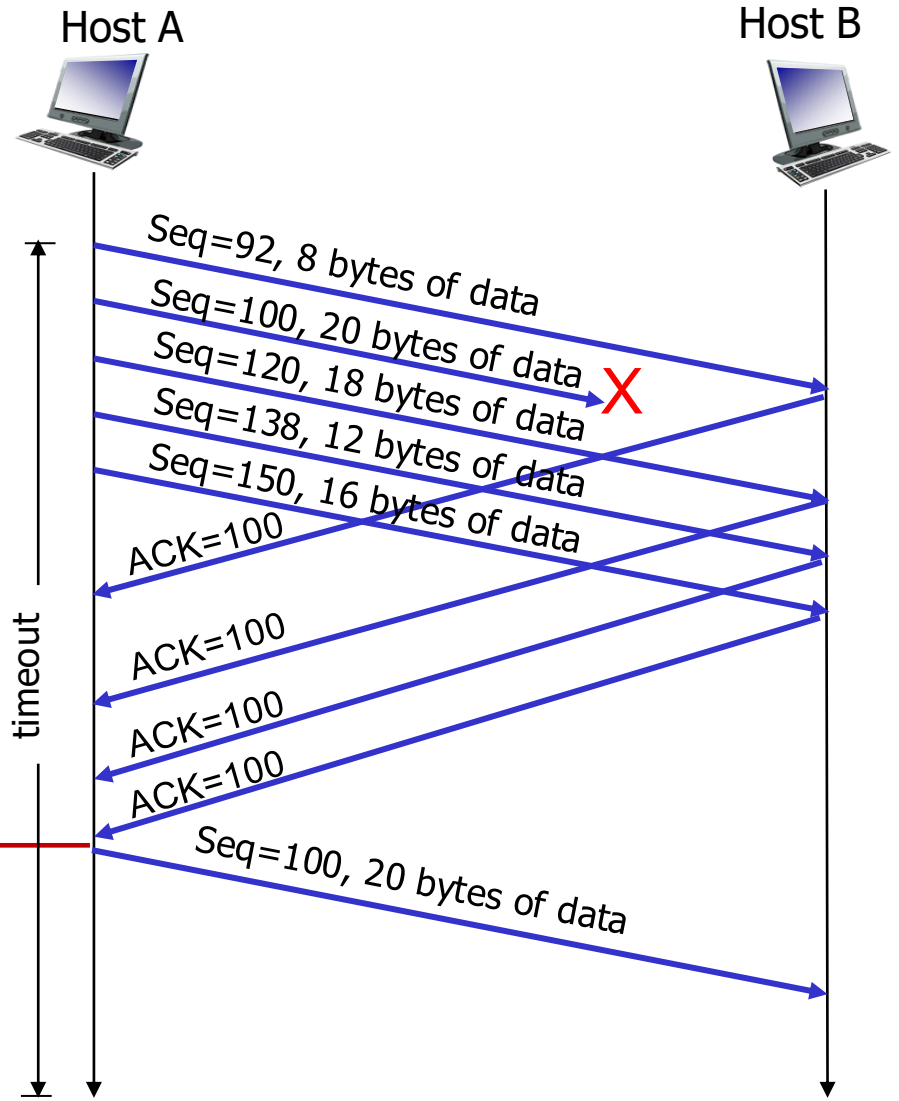
TCP fast retransmit

if sender receives 3 additional ACKs for same data ("triple duplicate ACKs"), resend unACKed segment with smallest seq #

- likely that unACKed segment lost, so don't wait for timeout



Receipt of three duplicate ACKs indicates 3 segments received after a missing segment – lost segment is likely. So retransmit!

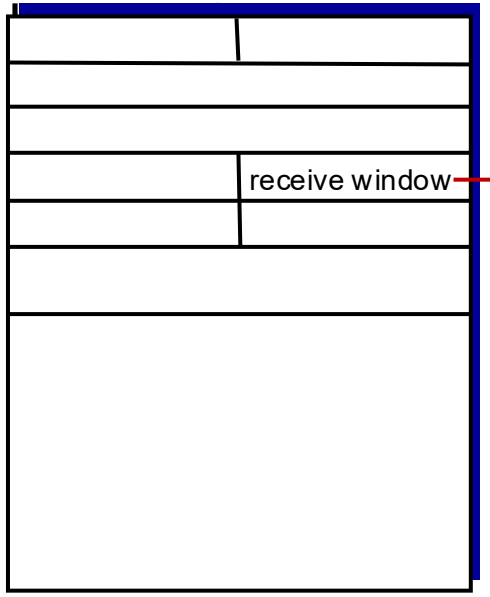


Lesson outline

- Transport-layer services
- Multiplexing and demultiplexing
- Connectionless transport: UDP
- Connection-oriented transport: TCP
- TCP connection management
- TCP Sequence numbers and acknowledgements
- TCP Retransmission
- TCP Flow control
- TCP congestion control

TCP flow control

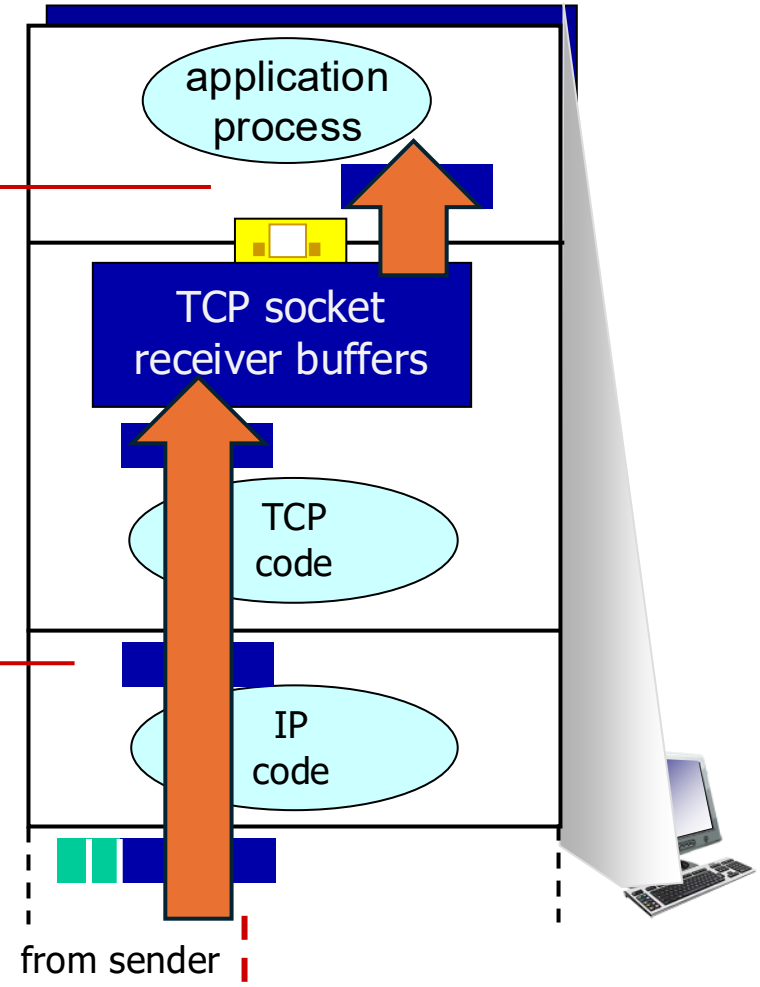
Q: What happens if network layer delivers data faster than application layer removes data from socket



rwnd : # bytes receiver willing to accept

Application removing data from TCP socket buffers

Network layer delivering IP datagram payload into TCP socket buffers



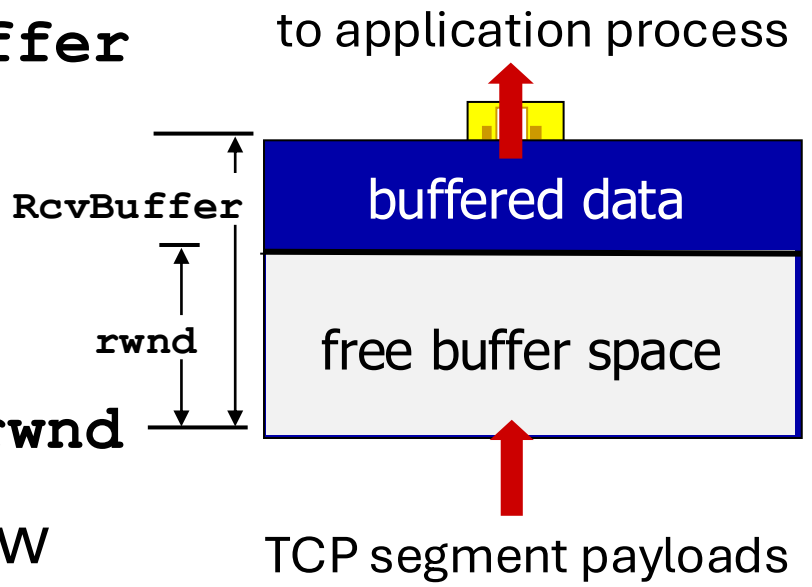
receiver protocol stack

flow control

receiver controls sender, so that sender won't overflow receiver's buffer by sending too much, too fast

TCP flow control

- TCP receiver "advertises" free buffer space in **`rwnd`** field in TCP header
  - **`RcvBuffer`** size set via socket options (typical default is 4096 bytes)
  - many operating systems auto-adjust **`RcvBuffer`**
  - Initially **`rwnd = RcvBuffer`**
- sender limits amount of unACKed ("in-flight") data to receiver's **`rwnd`**
  - **`LastByteSent - LastByteACKed`** $\leq$ **`rwnd`**
- guarantees receive buffer will not overflow



TCP receiver-side buffering

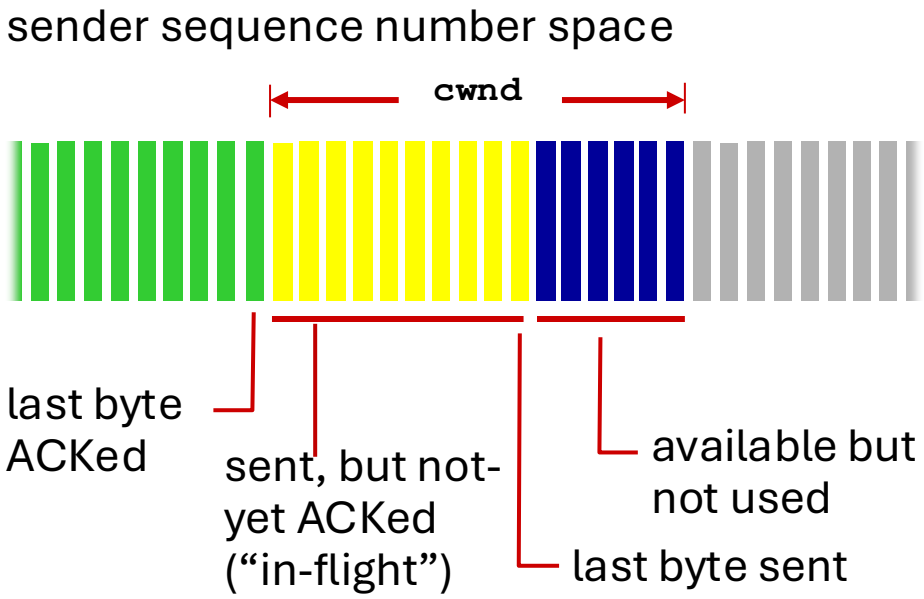
Lesson outline

- Transport-layer services
- Multiplexing and demultiplexing
- Connectionless transport: UDP
- Connection-oriented transport: TCP
- TCP connection management
- TCP Sequence numbers and acknowledgements
- TCP Retransmission
- TCP Flow control
- TCP congestion control

TCP Congestion control

- Many algorithms used
  - Tahoe, Reno, New Reno, BIC, CUBIC, …
- TCP Tahoe
  - Slow start + AIMD + Fast Retransmit
- TCP Reno
  - TCP Tahoe + Fast recovery

TCP congestion control: details



TCP sending behavior:

- roughly: send cwnd bytes, wait RTT for ACKS, then send more bytes

$$\text{TCP rate} \approx \frac{\text{cwnd}}{\text{RTT}} \text{ bytes/sec}$$

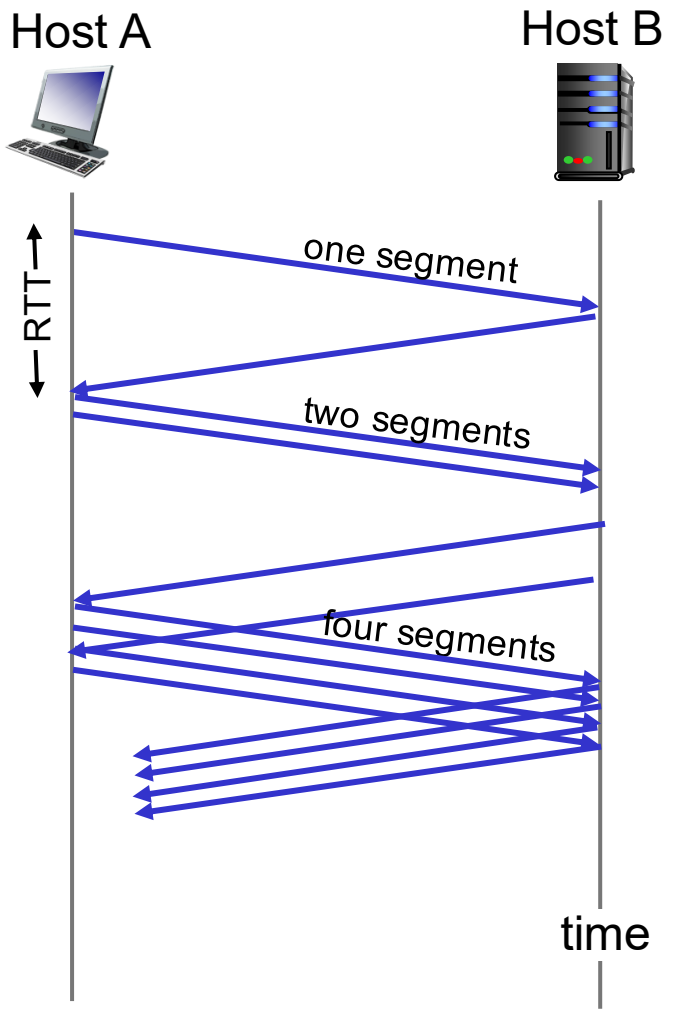
- TCP sender limits transmission:

$$\text{LastByteSent} - \text{LastByteACKed} \leq \text{cwnd}$$

- cwnd is dynamically adjusted in response to observed network congestion (implementing TCP congestion control)

TCP slow start

- when connection begins, increase rate exponentially until first loss event:
  - initially cwnd = 1 MSS
  - double cwnd every RTT
  - done by incrementing cwnd for every ACK received
- summary: initial rate is slow, but ramps up exponentially fast



TCP congestion control: AIMD

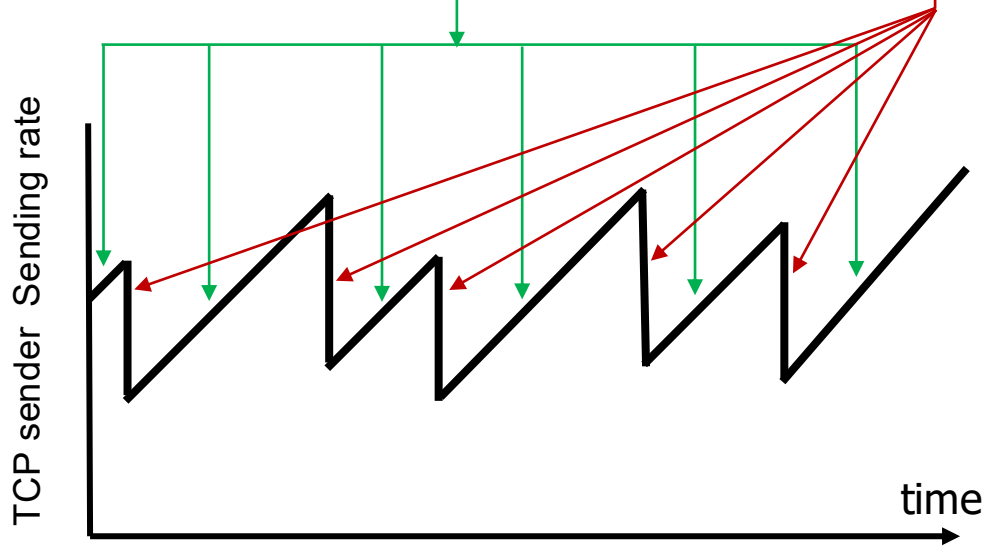
- approach: senders can increase cwnd until packet loss (congestion) occurs, then decrease sending rate on loss event

Additive Increase

increase cwnd by 1 maximum segment size every RTT until loss detected

Multiplicative Decrease

cut cwnd in half at each loss event



AIMD sawtooth behavior: probing for bandwidth

TCP AIMD: more

Multiplicative decrease detail: cwnd is

- Cut in half on loss detected by triple duplicate ACK (TCP Reno)
- Cut to 1 MSS (maximum segment size) when loss detected by timeout (TCP Tahoe)

Why AIMD?

- AIMD has been shown to:
  - optimize congested flow rates network wide!
  - have desirable stability properties

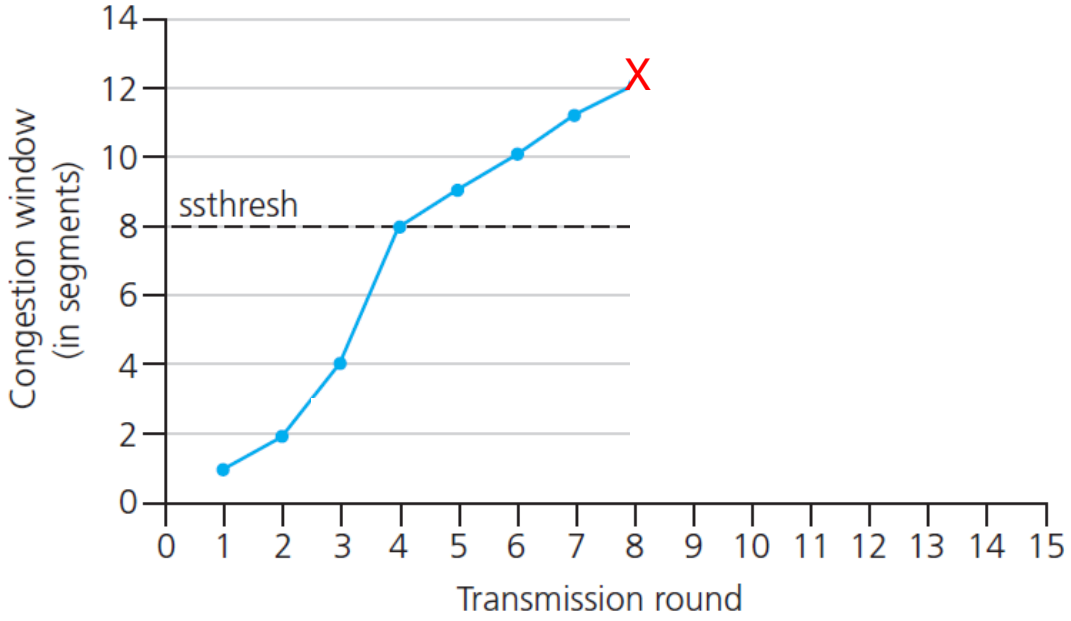
TCP: from slow start to congestion avoidance

Q: when should the exponential increase switch to linear?

A: when cwnd gets to 1/2 of its value before timeout.

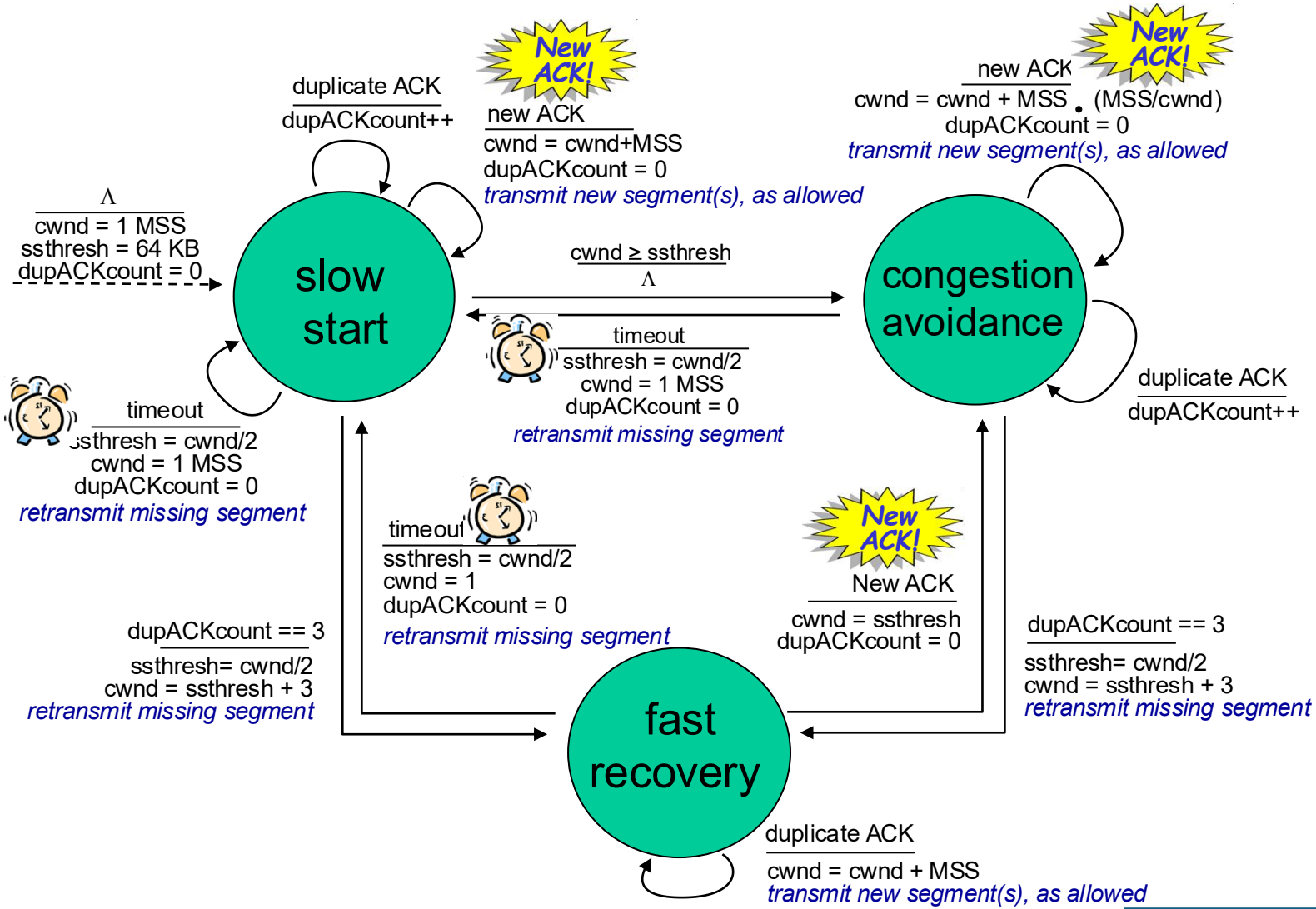
Implementation:

- variable ssthresh
- on loss event, ssthresh is set to 1/2 of cwnd just before loss event



* Check out the online interactive exercises for more examples: http://gaia.cs.umass.edu/kurose_ross/interactive/

Summary: TCP congestion control



TCP: Evolution of Congestion Control & Loss Recovery

- Pre-1988: Original TCP
  - No congestion control
  - Retransmission mainly based on timeout
- TCP Tahoe (1988) – CC+LR
  - Slow Start
  - Congestion Avoidance (AIMD)
  - Fast Retransmit
  - After loss:
    - ssthresh = previous cwnd / 2
    - cwnd = 1 MSS
- TCP Reno (1990) – CC+LR
  - Fast Recovery
  - After 3 duplicate ACKs:
    - ssthresh = previous cwnd / 2
    - cwnd ≈ ssthresh + 3 MSS
  - Avoids returning to cwnd = 1 for a single loss

TCP: Evolution of Congestion Control & Loss Recovery

- TCP SACK (1996) – LR
  - Selective Acknowledgment
  - Receiver reports blocks of data already received
  - More efficient recovery from multiple losses
- TCP NewReno (1999) – LR improvement
  - Improved Fast Recovery
  - Uses partial ACKs to recover multiple losses
  - Remains in Fast Recovery until the original window is recovered
- BIC TCP (2004) - CC
  - Binary Increase Congestion Control
  - Designed for high-speed/high-BDP networks
  - Aggressive growth combined with binary-search probing
- CUBIC (2008) – CC
  - Successor to BIC
  - Cubic congestion-window growth
  - Less dependent on RTT
  - Default congestion-control algorithm in Linux for many years

TCP: Evolution of Congestion Control & Loss Recovery

- RACK (2017) – Loss detection / LR
  - Recent ACKnowledgment
  - Time-based loss detection
  - Less dependent on duplicate-ACK counting
  - Handles reordering, multiple losses and tail losses better
- BBR (2016/2017) - CC
  - Bottleneck Bandwidth and Round-trip propagation time
  - Model-based congestion control
  - Estimates:
    - Bottleneck bandwidth
    - Minimum RTT
  - Controls sending rate based on estimated network capacity rather than primarily reacting to packet loss

References

- Chapter 3 - The Transport Layer
  James F. Kurose and Keith W. Ross, Computer Networking – A Top-Down Approach, (8th Edition), Pearson, 2020
- Chapter 15 - Transport Protocols
  William Stallings, Data and Computer Communications (10th Edition), Pearson, 2014

Lecture slides prepared by

Nimal Ratnayake
Dept of Computer Systems Engineering
SLIIT

First version: 2024 July
Revision 1: 2025 February
Revision 2: 2025 July
Revision 3: 2026 July

Do not edit
How to change the design



Attendance 2026-08-18

ⓘ The Slido app must be installed on every computer you're presenting from

slido

Do not edit
How to change the design



Attendance 2026-08-25

ⓘ The Slido app must be installed on every computer you're presenting from

slido

Do not edit
How to change the design



Audience Q&A

ⓘ The Slido app must be installed on every computer you're presenting from

slido